

C++ Standard Library Ready Issues to be moved in Kona, Nov. 2025

Doc. no. P3905R0

Date: 2025-10-30

Audience: WG21

Reply to: Jonathan Wakely <lwgchair@gmail.com>

- [Ready Issues](#)
 - [4137](#). Fix *Mandates*, *Preconditions*, and *Complexity* elements of [linalg] algorithms
 - [4243](#). `as_bytes/as_writable_bytes` is broken with `span<volatile T>`
 - [4315](#). Insufficient specification of `vector_two_norm` and `matrix_frob_norm`
 - [4423](#). `meta::data_member_spec` allows negative bit-field widths
- [Tentatively Ready Issues](#)
 - [2991](#). variant copy constructor missing `noexcept` (*see below*)
 - [3090](#). What is `[time.duration.cons]p4`'s "no overflow is induced in the conversion" intended to mean?
 - [3627](#). Inconsistent specifications for `std::make_optional` overloads
 - [4020](#). `extents::index-cast` weirdness
 - [4136](#). Specify behavior of [linalg] Hermitian algorithms on diagonal with nonzero imaginary part
 - [4166](#). `concat_view::end()` should be more constrained in order to support noncopyable iterators
 - [4253](#). `basic_const_iterator` should provide `iterator_type`
 - [4255](#). `move_only_function` constructor should recognize empty copyable functions
 - [4256](#). Incorrect constrains for `function_ref` constructors from `nontype_t`
 - [4257](#). Stream insertion for `chrono::local_time` should be constrained
 - [4265](#). `std::midpoint` should not accept `const bool`
 - [4266](#). `layout_stride::mapping` should treat empty mappings as exhaustive
 - [4269](#). `unique_copy` passes arguments to its predicate backwards
 - [4274](#). The `chrono::hh_mm_ss` constructor is ill-formed for unsigned durations
 - [4275](#). `std::dynamic_extent` should also be defined in `<mdspan>`
 - [4276](#). `front()` and `back()` are not hardened for zero-length `std::arrays`
 - [4280](#). `simd::partial_load` uses undefined identifier `T`
 - [4286](#). Some more feature-test macros for fully freestanding features are not marked freestanding
 - [4291](#). `explicit map(const Allocator&)` should be `constexpr`
 - [4292](#). Unordered container local iterators should be `constexpr` iterators
 - [4293](#). `span::subspan/first/last` chooses wrong constructor when `T` is const-qualified `bool`
 - [4294](#). `bitset(const CharT*)` constructor needs to be constrained
 - [4297](#). Missing *permutable* constraint for iterator overloads in Parallel Range Algorithms
 - [4299](#). Missing *Mandates*: part in `optional<T>::transform`
 - [4300](#). Missing *Returns*: element in `optional<T>::emplace`
 - [4301](#). `condition_variable_any::wait_for, until` should take timeout by value
 - [4305](#). Missing user requirements on `type_order` template
 - [4312](#). Const and value category mismatch for `allocator_arg_t/allocator_arg` in the description of uses-allocator construction
 - [4317](#). The meaning of "resource" in the *Cpp17Destructible* requirements is undefined
 - [4318](#). Have `hive::erase_if` reevaluate `end()` to avoid UB
 - [4328](#). Remove note in `[exec.sched]` regarding waiting for completion of scheduled operations
 - [4340](#). `task::promise_type::unhandled_stopped()` should be `noexcept`
 - [4341](#). Missing rvalue reference qualification for `task::connect()`
 - [4342](#). Missing rvalue reference qualification for `task_scheduler::ts-sender::connect()`
 - [4343](#). Missing default template arguments for `task`
 - [4345](#). `task::promise_type::return_value` default template parameter
 - [4346](#). `task::promise_type::return_void / value` lack a specification
 - [4349](#). `task` is not actually started lazily
 - [4351](#). *integral-constant-like* needs more `remove_cvref_t`
 - [4366](#). Heterogeneous comparison of expected may be ill-formed
 - [4370](#). Comparison of `optional<T>` to `T` may be ill-formed
 - [4372](#). Weaken *Mandates*: for dynamic padding values in padded layouts
 - [4375](#). `std::simd::bit_ceil` should not be `noexcept`
 - [4377](#). Misleading note about lock-free property of `std::atomic_ref`
 - [4382](#). The `simd::basic_mask(bool)` overload needs to be more constrained
 - [4384](#). `flat_set::erase(iterator)` is underconstrained
 - [4398](#). `enable_nonlocking_formatter_optimization` should be disabled for container adaptors
 - [4399](#). `enable_nonlocking_formatter_optimization` for pair and tuple needs `remove_cvref_t`
 - [4403](#). `simd::basic_vec` CTAD misses difference type casting
 - [4407](#). *constexpr-wrapper-like* needs `remove_cvref_t` in `simd::basic_vec` constructor
 - [4412](#). Fix declaration of `zero_element` and `uninit_element`
 - [4413](#). Unused/left-over `simd::alignment` specialization for `basic_mask`
 - [4415](#). `task::promise_type::uncaught_exception` seems to be misnamed
 - [4416](#). `<meta>` should include `<compare>`
 - [4422](#). `meta::access_context` should be a consteval-only type
 - [4425](#). CTAD `function_ref` of data member pointer should produce `noexcept` signature
 - [4426](#). Clarify what `meta::reflect_constant_string` considers a string literal

Ready Issues

[4137](#). Fix *Mandates*, *Preconditions*, and *Complexity* elements of [linalg] algorithms

Section: 29.9.14 [[linalg.algs.blas2](#)], 29.9.15 [[linalg.algs.blas3](#)] Status: [Voting](#) Submitter: Mark Hoemmen Opened: 2024-08-08 Last modified: 2025-10-30

Priority: Not Prioritized

Discussion:

Addresses [US 172-275](#)

As [pointed out by Raffaele Solcà](#) (CSCS Swiss National Supercomputing Centre), some of the *Mandates*, *Preconditions*, and *Complexity* elements of some BLAS 2 and BLAS 3 algorithms in [linalg] are incorrect.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4988](#).

1. Modify 29.9.14.1 [[linalg.algs.blas2.gemv](#)] as indicated:

[Drafting note: This change is needed because the matrix A does not need to be square. `x.extents(0)` must equal `A.extents(1)`, while `y.extents(0)` must equal `A.extents(0)`.]

-3- *Mandates*:

(3.1) — *possibly-multipliable*<decltype(A), decltype(x), decltype(y)>() is true, and

(3.2) — *possibly-addable*<decltype(~~y~~~~*~~), decltype(y), decltype(z)>() is true for those overloads that take a z parameter.

-4- *Preconditions*:

(4.1) — *multipliable*(A, x, y) is true, and

(4.2) — *addable*(~~y~~~~*~~, y, z) is true for those overloads that take a z parameter.

-5- *Complexity*: $O(\text{A}.\text{extent}(0) \times \text{x}.\text{A}.\text{extent}(\text{0}\pm))$.

2. Modify 29.9.14.2 [[linalg.algs.blas2.symv](#)] as indicated:

-3- *Mandates*:

(3.1) — [...]

(3.2) — [...]

(3.3) — *possibly-multipliable*<decltype(A), decltype(x), decltype(y)>() is true, and

(3.4) — *possibly-addable*<decltype(~~y~~~~*~~), decltype(y), decltype(z)>() is true for those overloads that take a z parameter.

-4- *Preconditions*:

(4.1) — `A.extent(0)` equals `A.extent(1)`,

(4.2) — *multipliable*(A, x, y) is true, and

(4.3) — *addable*(~~y~~~~*~~, y, z) is true for those overloads that take a z parameter.

-5- *Complexity*: $O(\text{A}.\text{extent}(0) \times \text{x}.\text{A}.\text{extent}(\text{0}\pm))$.

3. Modify 29.9.14.3 [[linalg.algs.blas2.hemv](#)] as indicated:

-3- *Mandates*:

(3.1) — [...]

(3.2) — [...]

(3.3) — *possibly-multipliable*<decltype(A), decltype(x), decltype(y)>() is true, and

(3.4) — *possibly-addable*<decltype(~~y~~~~*~~), decltype(y), decltype(z)>() is true for those overloads that take a z parameter.

-4- Preconditions:

- (4.1) — $A.\text{extent}(0)$ equals $A.\text{extent}(1)$,
- (4.2) — $\text{multipliable}(A, x, y)$ is true, and
- (4.3) — $\text{addable}(y, y, z)$ is true for those overloads that take a z parameter.

-5- Complexity: $\mathcal{O}(A.\text{extent}(0) \times x.A.\text{extent}(0+1))$.

4. Modify 29.9.14.4 [linalg.algs.blas2.trmv] as indicated:

[Drafting note: The extents compatibility conditions are expressed differently than in the above matrix-vector multiply sections, perhaps more for consistency with the TRSV section below. They look correct here. The original *Complexity* elements adjusted below are technically correct, since A is square, but changing this would improve consistency with 29.9.14.1 [linalg.algs.blas2.gemv]]

```
template<in-matrix InMat, class Triangle, class DiagonalStorage, in-vector InVec,
        out-vector OutVec>
void triangular_matrix_vector_product(InMat A, Triangle t, DiagonalStorage d, InVec x, OutVec y);
template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage, in-vector InVec,
        out-vector OutVec>
void triangular_matrix_vector_product(ExecutionPolicy&& exec,
                                     InMat A, Triangle t, DiagonalStorage d, InVec x, OutVec y);
```

-5- [...]

-6- Effects: Computes $y = Ax$.

-5- Complexity: $\mathcal{O}(A.\text{extent}(0) \times x.A.\text{extent}(0+1))$.

```
template<in-matrix InMat, class Triangle, class DiagonalStorage, inout-vector InOutVec>
void triangular_matrix_vector_product(InMat A, Triangle t, DiagonalStorage d, InOutVec y);
template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage, inout-vector InOutVec>
void triangular_matrix_vector_product(ExecutionPolicy&& exec,
                                     InMat A, Triangle t, DiagonalStorage d, InOutVec y);
```

-8- [...]

-9- Effects: [...]

-10- Complexity: $\mathcal{O}(A.y.\text{extent}(0) \times y.A.\text{extent}(0+1))$.

```
template<in-matrix InMat, class Triangle, class DiagonalStorage,
        in-vector InVec1, in-vector InVec2, out-vector OutVec>
void triangular_matrix_vector_product(InMat A, Triangle t, DiagonalStorage d,
                                     InVec1 x, InVec2 y, OutVec z);
template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage,
        in-vector InVec1, in-vector InVec2, out-vector OutVec>
void triangular_matrix_vector_product(ExecutionPolicy&& exec,
                                     InMat A, Triangle t, DiagonalStorage d,
                                     InVec1 x, InVec2 y, OutVec z);
```

-11- [...]

-12- Effects: Computes $z = y + Ax$.

-13- Complexity: $\mathcal{O}(A.\text{extent}(0) \times x.A.\text{extent}(0+1))$.

5. Modify 29.9.15.4 [linalg.algs.blas3.rankk] as indicated:

[Drafting note: P3371R0, to be submitted in the August 15 mailing for LEWG review, contains the same wording changes to 29.9.15.4 [linalg.algs.blas3.rankk] and 29.9.15.5 [linalg.algs.blas3.rank2k] as proposed here, with additional changes corresponding to that proposal. Please apply this LWG issue's changes first, before P3371 merges]

-3- Mandates:

(3.1) — If InOutMat has layout_blas_packed layout, then the layout's Triangle template argument has the same type as the function's Triangle template argument; and

(3.2) — ~~$\text{possibly-multipliable}(\text{decltype}(A), \text{decltype}(\text{transposed}(A)), \text{decltype}(C))$~~ ~~$\text{compatible-static-extents}(\text{decltype}(A), \text{decltype}(A)) > (0, 1)$~~ is true; ~~and~~

~~(3.3) — $\text{compatible-static-extents}(\text{decltype}(C), \text{decltype}(C)) > (0, 1)$ is true; and~~

~~(3.4) — $\text{compatible-static-extents}(\text{decltype}(A), \text{decltype}(C)) > (0, 0)$ is true;~~

-4- Preconditions: `multipliable(A, transposed(A), C)` is true.

(4.1) — `A.extent(0)` equals `A.extent(1)`.

(4.2) — `C.extent(0)` equals `C.extent(1)`, and

(4.3) — `A.extent(0)` equals `C.extent(0)`.

-5- Complexity: $\mathcal{O}(A.\text{extent}(0) \times A.\text{extent}(1) \times A.C.\text{extent}(0))$.

6. Modify 29.9.15.5 [linalg.algs.blas3.rank2k] as indicated:

-3- Mandates:

(3.1) — If `InOutMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;

(3.2) — `possibly-multipliable<decltype(A), decltype(transposed(B)), decltype(C)>()` `possibly-addable<decltype(A), decltype(B), decltype(C)>()` is true; and

(3.3) — `possibly-multipliable<decltype(B), decltype(transposed(A)), decltype(C)>(0, 1)` `compatible static extents<decltype(A), decltype(A)>(0, 1)` is true.

-4- Preconditions:

(4.1) — `multipliable(A, transposed(B), C)` `addable(A, B, C)` is true, and

(4.2) — `multipliable(B, transposed(A), C)` is true `A.extent(0)` equals `A.extent(1)`.

-5- Complexity: $\mathcal{O}(A.\text{extent}(0) \times A.\text{extent}(1) \times B.C.\text{extent}(0))$.

7. Modify 29.9.15.6 [linalg.algs.blas3.trsm] as indicated:

[Drafting note: Nothing is wrong here, but it's nice to make the complexity clauses depend only on input if possible]

```
template<in-matrix InMat1, class Triangle, class DiagonalStorage,
        in-matrix InMat2, out-matrix OutMat, class BinaryDivideOp>
void triangular_matrix_matrix_left_solve(InMat1 A, Triangle t, DiagonalStorage d,
                                         InMat2 B, OutMat X, BinaryDivideOp divide);
template<class ExecutionPolicy,
        in-matrix InMat1, class Triangle, class DiagonalStorage,
        in-matrix InMat2, out-matrix OutMat, class BinaryDivideOp>
void triangular_matrix_matrix_left_solve(ExecutionPolicy&& exec,
                                         InMat1 A, Triangle t, DiagonalStorage d,
                                         InMat2 B, OutMat X, BinaryDivideOp divide);
```

[...]

-6- Complexity: $\mathcal{O}(A.\text{extent}(0) \times B.X.\text{extent}(1) \times B.X.\text{extent}(1))$.

8. Modify 29.9.15.7 [linalg.algs.blas3.inplacestrsm] as indicated:

[Drafting note: Nothing is wrong here, but it's nice to make the complexity clauses depend only on input if possible]

```
template<in-matrix InMat, class Triangle, class DiagonalStorage,
        inout-matrix InOutMat, class BinaryDivideOp>
void triangular_matrix_matrix_right_solve(InMat A, Triangle t, DiagonalStorage d,
                                         InOutMat B, BinaryDivideOp divide);
template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage,
        inout-matrix InOutMat, class BinaryDivideOp>
void triangular_matrix_matrix_right_solve(ExecutionPolicy&& exec,
                                         InMat A, Triangle t, DiagonalStorage d,
                                         InOutMat B, BinaryDivideOp divide);
```

[...]

-13- Complexity: $\mathcal{O}(B.A.\text{extent}(0) \times A.\text{extent}(0+1) \times A.B.\text{extent}(1))$.

[LWG telecon 2025-10-10; Fix proposed resolution after review]

Add missing () in one place and change (0, 1) to () in another.

[LWG telecon 2025-10-10; Status updated New → Ready]

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 29.9.14.1 [\[linalg.blas2.gemv\]](#) as indicated:

[Drafting note: This change is needed because the matrix A does not need to be square. `x.extents(0)` must equal `A.extents(1)`, while `y.extents(0)` must equal `A.extents(0)`.]

-3- Mandates:

(3.1) — *possibly-multiplicable*<decltype(A), decltype(x), decltype(y)>() is true, and

(3.2) — *possibly-addable*<decltype(~~y~~x), decltype(y), decltype(z)>() is true for those overloads that take a z parameter.

-4- Preconditions:

(4.1) — *multiplicable*(A, x, y) is true, and

(4.2) — *addable*(~~y~~x, y, z) is true for those overloads that take a z parameter.

-5- Complexity: $\mathcal{O}(A.x.extent(0) \times x.A.extent(0 \pm 1))$.

2. Modify 29.9.14.2 [\[linalg.blas2.sylv\]](#) as indicated:

-3- Mandates:

(3.1) — [...]

(3.2) — [...]

(3.3) — *possibly-multiplicable*<decltype(A), decltype(x), decltype(y)>() is true, and

(3.4) — *possibly-addable*<decltype(~~y~~x), decltype(y), decltype(z)>() is true for those overloads that take a z parameter.

-4- Preconditions:

(4.1) — `A.extent(0)` equals `A.extent(1)`,

(4.2) — *multiplicable*(A, x, y) is true, and

(4.3) — *addable*(~~y~~x, y, z) is true for those overloads that take a z parameter.

-5- Complexity: $\mathcal{O}(A.x.extent(0) \times x.A.extent(0 \pm 1))$.

3. Modify 29.9.14.3 [\[linalg.blas2.hemv\]](#) as indicated:

-3- Mandates:

(3.1) — [...]

(3.2) — [...]

(3.3) — *possibly-multiplicable*<decltype(A), decltype(x), decltype(y)>() is true, and

(3.4) — *possibly-addable*<decltype(~~y~~x), decltype(y), decltype(z)>() is true for those overloads that take a z parameter.

-4- Preconditions:

(4.1) — `A.extent(0)` equals `A.extent(1)`,

(4.2) — *multiplicable*(A, x, y) is true, and

(4.3) — *addable*(~~y~~x, y, z) is true for those overloads that take a z parameter.

-5- Complexity: $\mathcal{O}(A.x.extent(0) \times x.A.extent(0 \pm 1))$.

4. Modify 29.9.14.4 [\[linalg.blas2.trmv\]](#) as indicated:

[Drafting note: The extents compatibility conditions are expressed differently than in the above matrix-vector multiply sections, perhaps more for consistency with the TRSV section below. They look correct here. The original *Complexity* elements adjusted below are technically correct, since *A* is square, but changing this would improve consistency with 29.9.14.1 [\[linalg.blas2.gemv\]](#)]

```
template<in-matrix InMat, class Triangle, class DiagonalStorage, in-vector InVec,
        out-vector OutVec>
void triangular_matrix_vector_product(InMat A, Triangle t, DiagonalStorage d, InVec x, OutVec y);
template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage, in-vector InVec,
        out-vector OutVec>
```

```
void triangular_matrix_vector_product(ExecutionPolicy&& exec,
                                     InMat A, Triangle t, DiagonalStorage d, InVec x, OutVec y);
```

-5- [...]

-6- *Effects*: Computes $y = Ax$.

-5- *Complexity*: $\mathcal{O}(A_{**}.extent(0) \times xA.extent(0 \pm))$.

```
template<in-matrix InMat, class Triangle, class DiagonalStorage, inout-vector InOutVec>
void triangular_matrix_vector_product(InMat A, Triangle t, DiagonalStorage d, InOutVec y);
template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage, inout-vector InOutVec>
void triangular_matrix_vector_product(ExecutionPolicy&& exec,
                                     InMat A, Triangle t, DiagonalStorage d, InOutVec y);
```

-8- [...]

-9- *Effects*: [...]

-10- *Complexity*: $\mathcal{O}(Ay.extent(0) \times yA.extent(0 \pm))$.

```
template<in-matrix InMat, class Triangle, class DiagonalStorage,
        in-vector InVec1, in-vector InVec2, out-vector OutVec>
void triangular_matrix_vector_product(InMat A, Triangle t, DiagonalStorage d,
                                     InVec1 x, InVec2 y, OutVec z);
template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage,
        in-vector InVec1, in-vector InVec2, out-vector OutVec>
void triangular_matrix_vector_product(ExecutionPolicy&& exec,
                                     InMat A, Triangle t, DiagonalStorage d,
                                     InVec1 x, InVec2 y, OutVec z);
```

-11- [...]

-12- *Effects*: Computes $z = y + Ax$.

-13- *Complexity*: $\mathcal{O}(A_{**}.extent(0) \times xA.extent(0 \pm))$.

5. Modify 29.9.15.4 [\[linalg.blas3.rankk\]](#) as indicated:

[Drafting note: P3371R0, to be submitted in the August 15 mailing for LEWG review, contains the same wording changes to 29.9.15.4 [\[linalg.blas3.rankk\]](#) and 29.9.15.5 [\[linalg.blas3.rank2k\]](#) as proposed here, with additional changes corresponding to that proposal. Please apply this LWG issue's changes first, before P3371 merges]

-3- *Mandates*:

(3.1) — If InOutMat has layout_blas_packed layout, then the layout's Triangle template argument has the same type as the function's Triangle template argument; and

(3.2) — possibly-multipliable<decltype(A), decltype(transposed(A)), decltype(C)>() ~~compatible-static-extents<decltype(A), decltype(A)>(0, 1)~~ is true; ~~and~~

~~(3.3) — compatible-static-extents<decltype(C), decltype(C)>(0, 1) is true; and~~

~~(3.4) — compatible-static-extents<decltype(A), decltype(C)>(0, 0) is true.~~

-4- *Preconditions*: multipliable(A, transposed(A), C) is true.

~~(4.1) — A.extent(0) equals A.extent(1);~~

~~(4.2) — C.extent(0) equals C.extent(1); and~~

~~(4.3) — A.extent(0) equals C.extent(0);~~

-5- *Complexity*: $\mathcal{O}(A.extent(0) \times A.extent(1) \times A.extent(0))$.

6. Modify 29.9.15.5 [\[linalg.blas3.rank2k\]](#) as indicated:

-3- *Mandates*:

(3.1) — If InOutMat has layout_blas_packed layout, then the layout's Triangle template argument has the same type as the function's Triangle template argument;

(3.2) — possibly-multipliable<decltype(A), decltype(transposed(B)), decltype(C)>() ~~possibly-addable<decltype(A), decltype(B), decltype(C)>()~~ is true; and

(3.3) — possibly-multipliable<decltype(B), decltype(transposed(A)), decltype(C)>() ~~compatible-static-extents<decltype(A), decltype(A)>(0, 1)~~ is true.

-4- Preconditions:

(4.1) — `multipliable(A, transposed(B), C)` `addable(A, B, C)` is true, and

(4.2) — `multipliable(B, transposed(A), C)` is true `A.extent(0) equals A.extent(1)`.

-5- Complexity: $\mathcal{O}(A.extent(0) \times A.extent(1) \times B.extent(0))$.

7. Modify 29.9.15.6 [\[linalg.blas3.trsm\]](#) as indicated:

[Drafting note: Nothing is wrong here, but it's nice to make the complexity clauses depend only on input if possible]

```
template<in-matrix InMat1, class Triangle, class DiagonalStorage,
        in-matrix InMat2, out-matrix OutMat, class BinaryDivideOp>
void triangular_matrix_matrix_left_solve(InMat1 A, Triangle t, DiagonalStorage d,
                                         InMat2 B, OutMat X, BinaryDivideOp divide);
template<class ExecutionPolicy,
        in-matrix InMat1, class Triangle, class DiagonalStorage,
        in-matrix InMat2, out-matrix OutMat, class BinaryDivideOp>
void triangular_matrix_matrix_left_solve(ExecutionPolicy&& exec,
                                         InMat1 A, Triangle t, DiagonalStorage d,
                                         InMat2 B, OutMat X, BinaryDivideOp divide);

[...]
```

-6- Complexity: $\mathcal{O}(A.extent(0) \times B.extent(1) \times B.extent(1))$.

8. Modify 29.9.15.7 [\[linalg.blas3.inplace_trsm\]](#) as indicated:

[Drafting note: Nothing is wrong here, but it's nice to make the complexity clauses depend only on input if possible]

```
template<in-matrix InMat, class Triangle, class DiagonalStorage,
        inout-matrix InOutMat, class BinaryDivideOp>
void triangular_matrix_matrix_right_solve(InMat A, Triangle t, DiagonalStorage d,
                                          InOutMat B, BinaryDivideOp divide);
template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage,
        inout-matrix InOutMat, class BinaryDivideOp>
void triangular_matrix_matrix_right_solve(ExecutionPolicy&& exec,
                                          InMat A, Triangle t, DiagonalStorage d,
                                          InOutMat B, BinaryDivideOp divide);

[...]
```

-13- Complexity: $\mathcal{O}(B.extent(0) \times A.extent(0) \times A.extent(1))$.

4243. `as_bytes/as_writable_bytes` is broken with `span<volatile T>`

Section: 23.7.2.3 [\[span.objectrep\]](#) Status: [Voting](#) Submitter: Hewill Kang Opened: 2025-04-12 Last modified: 2025-10-30

Priority: 4

Discussion:

They both use `reinterpret_cast` to cast the underlying pointer type of `span` to `const byte*` and `byte*` respectively, which leads to a hard error when the element type is `volatile-qualified` ([demo](#)):

```
#include <span>

int main() {
    std::span<volatile int> span;
    auto bytes = as_bytes(span); // hard error
    auto writable_bytes = as_writable_bytes(span); // hard error
}
```

Previous resolution [SUPERSEDED]:

This wording is relative to [N5008](#).

1. Modify 23.7.2.1 [\[span.syn\]](#), header `<span>` synopsis, as indicated:

```
[...]
namespace std {
[...]
```

// 23.7.2.3 [\[span.objectrep\]](#), views of object representation

```
template<class ElementType, size_t Extent>
    span<const conditional_t<is_volatile_v<ElementType>, volatile byte, byte>,
        Extent == dynamic_extent ? dynamic_extent : sizeof(ElementType) * Extent>
```

```

    as_bytes(span<ElementType, Extent> s) noexcept;

template<class ElementType, size_t Extent>
span<conditional_t<is_volatile_v<ElementType>, volatile byte, byte>,
    Extent == dynamic_extent ? dynamic_extent : sizeof(ElementType) * Extent>
    as_writable_bytes(span<ElementType, Extent> s) noexcept;
}

```

2. Modify 23.7.2.3 [\[span.objectrep\]](#) as indicated:

```

template<class ElementType, size_t Extent>
span<const conditional_t<is_volatile_v<ElementType>, volatile byte, byte>,
    Extent == dynamic_extent ? dynamic_extent : sizeof(ElementType) * Extent>
    as_bytes(span<ElementType, Extent> s) noexcept;

-1- Effects: Equivalent to: return R{reinterpret_cast<R::pointer_const_byte*>(s.data()),
    s.size_bytes()};
    where R is the return type.

template<class ElementType, size_t Extent>
span<conditional_t<is_volatile_v<ElementType>, volatile byte, byte>,
    Extent == dynamic_extent ? dynamic_extent : sizeof(ElementType) * Extent>
    as_writable_bytes(span<ElementType, Extent> s) noexcept;

-2- Constraints: is_const_v<ElementType> is false.

-3- Effects: Equivalent to: return R{reinterpret_cast<R::pointer_byte*>(s.data()),
    s.size_bytes()};
    where R is the return type.

```

[2025-04-16; Hewill Kang provides alternative wording]

Based on reflector feedback, the revised wording just improves the current state of not supporting support for volatile.

[2025-06-12; Reflector poll]

Set priority to 4 after reflector poll.

[Sofia 2025-06-17; Move to Ready]

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 23.7.2.3 [\[span.objectrep\]](#) as indicated:

```

template<class ElementType, size_t Extent>
span<const byte, Extent == dynamic_extent ? dynamic_extent : sizeof(ElementType) * Extent>
    as_bytes(span<ElementType, Extent> s) noexcept;

-?- Constraints: is_volatile_v<ElementType> is false.

-1- Effects: Equivalent to: return R{reinterpret_cast<const byte*>(s.data()), s.size_bytes()};
    where R is the return type.

template<class ElementType, size_t Extent>
span<byte, Extent == dynamic_extent ? dynamic_extent : sizeof(ElementType) * Extent>
    as_writable_bytes(span<ElementType, Extent> s) noexcept;

-2- Constraints: is_const_v<ElementType> is false and is_volatile_v<ElementType> is false.

-3- Effects: Equivalent to: return R{reinterpret_cast<byte*>(s.data()), s.size_bytes()};
    where R is the return type.

```

4315. Insufficient specification of vector_two_norm and matrix_frob_norm

Section: 29.9.13.9 [\[linalg.algs.blas1_nrm2\]](#), 29.9.13.12 [\[linalg.algs.blas1_matfrobnorm\]](#) **Status:** [Voting](#) **Submitter:** Mark Hoemmen **Opened:** 2025-08-14 **Last modified:** 2025-10-30

Priority: Not Prioritized

Discussion:

Addresses [US 171-274](#)

The Returns clauses of vector_two_norm 29.9.13.9 [\[linalg.algs.blas1_nrm2\]](#) and matrix_frob_norm 29.9.13.12 [\[linalg.algs.blas1_matfrobnorm\]](#) say that the functions return the "square root" of the sum of squares of the initial value and the absolute values of the elements of the input mdspan. However, nowhere in 29.9 [\[linalg\]](#) explains how to compute a square root.

1. The input `mdspan`'s `value_type` and the initial value type are not constrained in a way that would ensure that calling `std::sqrt` on this expression would be well-formed.
2. There is no provision to find `sqrt` via argument-dependent lookup, even though 29.9 [linalg] has provisions to find `abs`, `conj`, `real`, and `imag` via argument-dependent lookup. There is no "*sqrt-if-needed*" analog to *abs-if-needed*, *conj-if-needed*, *real-if-needed*, and *imag-if-needed*.

The easiest fix for both issues is just to *Constrain* both `Scalar` and the input `mdspan`'s `value_type` to be floating-point numbers or specializations of `std::complex` for these two functions. This presumes that relaxing this *Constraint* and fixing the above two issues later would be a non-breaking change. If that is *not* the case, then I would suggest removing the two functions entirely.

Previous resolution [SUPERSEDED]:

This wording is relative to [N5014](#).

[Drafting note: As a drive-by fix the proposed wording adds a missing closing parentheses in 29.9.13.9 [linalg.algs.blas1.nrm2] p2.]

1. Modify 29.9.13.9 [linalg.algs.blas1.nrm2] as indicated:

```
template<in-vector InVec, class Scalar>
    Scalar vector_two_norm(InVec v, Scalar init);
template<class ExecutionPolicy, in-vector InVec, class Scalar>
    Scalar vector_two_norm(ExecutionPolicy&& exec, InVec v, Scalar init);
```

-1- [Note 1: [...] — end note]

~~-2- Constraints: InVec::value_type and Scalar are either a floating-point type, or a specialization of complex.~~

-2- Mandates: Let `a` be *abs-if-needed*(`declval<typename InVec::value_type>()`). Then, `decltype(init + a * a)` is convertible to `Scalar`.

-3- Returns: The square root of the sum of the square of `init` and the squares of the absolute values of the elements of `v`.

[Note 2: For `init` equal to zero, this is the Euclidean norm (also called 2-norm) of the vector `v`. — end note]

~~-4- Remarks: If InVec::value_type and Scalar are all floating-point types or specializations of complex, and if~~ `Scalar` has higher precision than `InVec::value_type`, then intermediate terms in the sum use `Scalar`'s precision or greater.

[Note 3: An implementation of this function for floating-point types `T` can use the `scaled_sum_of_squares` result from `vector_sum_of_squares(x, {scaling_factor=1.0, scaled_sum_of_squares=init})`. — end note]

2. Modify 29.9.13.12 [linalg.algs.blas1.matfrobnorm] as indicated:

```
template<in-matrix InMat, class Scalar>
    Scalar matrix_frob_norm(InMat A, Scalar init);
template<class ExecutionPolicy, in-matrix InMat, class Scalar>
    Scalar matrix_frob_norm(ExecutionPolicy&& exec, InMat A, Scalar init);
```

~~-2- Constraints: InVec::value_type and Scalar are either a floating-point type, or a specialization of complex.~~

-2- Mandates: Let `a` be *abs-if-needed*(`declval<typename InMat::value_type>()`). Then, `decltype(init + a * a)` is convertible to `Scalar`.

-3- Returns: The square root of the sum of squares of `init` and the absolute values of the elements of `A`.

[Note 2: For `init` equal to zero, this is the Frobenius norm of the matrix `A`. — end note]

~~-4- Remarks: If InMat::value_type and Scalar are all floating-point types or specializations of complex, and if~~ `Scalar` has higher precision than `InMat::value_type`, then intermediate terms in the sum use `Scalar`'s precision or greater.

[LWG telecon 2025-10-10; Update proposed resolution after review]

Use *Mandates*: for the new requirements, because we plan to change this later so want to make it ill-formed, not something that is statically checkable as part of the API.

[LWG telecon 2025-10-10; Status updated New → Ready]

Proposed resolution:

This wording is relative to [N5014](#).

[Drafting note: As a drive-by fix the proposed wording adds a missing closing parentheses in 29.9.13.9 [\[linalg.blas1.nrm2\]](#) p2.]

1. Modify 29.9.13.9 [\[linalg.blas1.nrm2\]](#) as indicated:

```
template<in-vector InVec, class Scalar>
    Scalar vector_two_norm(InVec v, Scalar init);
template<class ExecutionPolicy, in-vector InVec, class Scalar>
    Scalar vector_two_norm(ExecutionPolicy&& exec, InVec v, Scalar init);
```

-1- [Note 1: [...] — end note]

-2- Mandates: [InVec::value_type](#) and [Scalar](#) are either a floating-point type, or a specialization of [complex](#). Let a be *abs-if-needed*(`declval<typename InVec::value_type>()`). Then, `decltype(init + a * a)` is convertible to [Scalar](#).

-3- Returns: The square root of the sum of the square of `init` and the squares of the absolute values of the elements of `v`.

[Note 2: For `init` equal to zero, this is the Euclidean norm (also called 2-norm) of the vector `v`. — end note]

-4- Remarks: If ~~[InVec::value_type](#) and [Scalar](#) are all floating-point types or specializations of [complex](#), and if~~ [Scalar](#) has higher precision than [InVec::value_type](#), then intermediate terms in the sum use [Scalar](#)'s precision or greater.

[Note 3: An implementation of this function for floating-point types `T` can use the `scaled_sum_of_squares` result from `vector_sum_of_squares(x, {.scaling_factor=1.0, .scaled_sum_of_squares=init})`. — end note]

2. Modify 29.9.13.12 [\[linalg.blas1.matfrobnorm\]](#) as indicated:

```
template<in-matrix InMat, class Scalar>
    Scalar matrix_frob_norm(InMat A, Scalar init);
template<class ExecutionPolicy, in-matrix InMat, class Scalar>
    Scalar matrix_frob_norm(ExecutionPolicy&& exec, InMat A, Scalar init);
```

-2- Mandates: [InVec::value_type](#) and [Scalar](#) are either a floating-point type, or a specialization of [complex](#). Let a be *abs-if-needed*(`declval<typename InMat::value_type>()`). Then, `decltype(init + a * a)` is convertible to [Scalar](#).

-3- Returns: The square root of the sum of squares of `init` and the absolute values of the elements of `A`.

[Note 2: For `init` equal to zero, this is the Frobenius norm of the matrix `A`. — end note]

-4- Remarks: If ~~[InMat::value_type](#) and [Scalar](#) are all floating-point types or specializations of [complex](#), and if~~ [Scalar](#) has higher precision than [InMat::value_type](#), then intermediate terms in the sum use [Scalar](#)'s precision or greater.

4423. meta::data_member_spec allows negative bit-field widths

Section: 21.4.16 [\[meta.reflection.define.aggregate\]](#) Status: [Voting](#) Submitter: Jakub Jelinek Opened: 2025-10-20 Last modified: 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [\[meta.reflection.define.aggregate\]](#).

View all other [issues](#) in [\[meta.reflection.define.aggregate\]](#).

Discussion:

The `meta::data_member_spec` function doesn't restrict `options.bit_width` to be non-negative.

[2025-10-24; LWG telecon; Status changed: New → Ready.]

Poll to move issue to Ready: 10/0/0

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 21.4.16 [\[meta.reflection.define.aggregate\]](#) as indicated:

- (5.3) — if `options.name` does not contain a value, then `options.bit_width` contains a value;
- (5.4) — if `options.bit_width` contains a value V , then
 - (5.4.1) — `is_integral_type(type) || is_enum_type(type)` is true,
 - (5.4.2) — `options.alignment` does not contain a value,
 - (5.4.3) — `options.no_unique_address` is false, ~~and~~
 - [\(5.4.?\) — \$V\$ is not negative, and](#)
 - (5.4.4) — if V equals 0, then `options.name` does not contain a value; and
- (5.5) — if `options.alignment` contains a value, it is an alignment value (6.8.3 [\[basic.align\]](#)) not less than `alignment_of(type)`.

Tentatively Ready Issues

[2991](#). variant copy constructor missing noexcept (see below)

Section: 22.6.3.2 [\[variant.ctor\]](#) Status: [Voting](#) Submitter: Peter Dimov Opened: 2017-06-27 Last modified: 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [\[variant.ctor\]](#).

View all other [issues](#) in [\[variant.ctor\]](#).

Discussion:

The copy constructor of `std::variant` is not conditionally noexcept (I think it was in the original proposal.)

It should be, for two reasons: first, this would be consistent with the other three constructors

```
constexpr variant() noexcept(see below);  
  
variant(variant&&) noexcept(see below);  
  
template <class T>  
constexpr variant(T&&) noexcept(see below);
```

and second, `variant` itself makes use of `is_nothrow_copy_constructible`, so it's inconsistent for it to take a stance against it.

[2017-07 Toronto Tuesday PM issue prioritization]

Status to LEWG

[Wrocław 2024-11-18; LEWG approves the direction]

In [P0088R1](#) the copy constructor was conditionally noexcept in the synopsis, but not the detailed description. This was pointed out during LWG review in Jacksonville. The approved paper, [P008R3](#), doesn't have it in either place.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4659](#).

1. Edit 22.6.3 [\[variant.variant\]](#), class template `variant` synopsis, as indicated:

```
template <class... Types>  
class variant {  
public:  
    // 23.7.3.1, constructors  
    constexpr variant() noexcept(see below);  
    variant(const variant&) noexcept(see below);  
    variant(variant&&) noexcept(see below);  
    [...]  
};
```

2. Edit 22.6.3.2 [\[variant.ctor\]](#) as indicated:

```
variant(const variant& w) noexcept(see below);  
  
[...]
```

-8- Remarks: This function shall not participate in overload resolution unless `is_copy_constructible_v<Ti>` is true for all *i*. The expression inside noexcept is equivalent to the logical AND of `is_nothrow_copy_constructible_v<Ti>` for all *i*.

[2025-10-20; Jonathan provides updated wording]

[2025-10-23; Reflector poll.]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [P5014](#).

1. Edit 22.6.3 [\[variant.variant\]](#), class template `variant` synopsis, as indicated:

```
template <class... Types>  
class variant {  
public:  
    // 23.7.3.1, constructors  
    constexpr variant() noexcept(see below);  
    variant(const variant&) noexcept(see below);  
    variant(variant&&) noexcept(see below);  
    [...]  
};
```

2. Edit 22.6.3.2 [\[variant.ctor\]](#) as indicated:

```
variant(const variant& w) noexcept(see below);
```

```
[...]
```

-8- *Remarks:* This function is defined as deleted unless `is_copy_constructible_v<Ti>` is true for all *i*. [The exception specification is equivalent to the logical AND of `is\_nothrow\_copy\_constructible\_v<Ti>` for all *i*.](#)

[3090](#). What is §[\[time.duration.cons\]](#)p4's "no overflow is induced in the conversion" intended to mean?

Section: 30.5.2 [\[time.duration.cons\]](#) **Status:** [Voting](#) **Submitter:** Richard Smith **Opened:** 2018-03-22 **Last modified:** 2025-10-30

Priority: 3

View all other [issues](#) in [\[time.duration.cons\]](#).

Discussion:

30.5.2 [\[time.duration.cons\]](#) p4 says:

```
template<class Rep2, class Period2>
constexpr duration(const duration<Rep2, Period2>& d);
```

Remarks: This constructor shall not participate in overload resolution unless no overflow is induced in the conversion and `treat_as_floating_point_v<rep>` is true or both `ratio_divide<Period2, period::den>` is 1 and `treat_as_floating_point_v<Rep2>` is false.

with this example:

```
duration<int, milli> ms(3);
duration<int, micro> us = ms; // OK
duration<int, milli> ms2 = us; // error
```

It's unclear to me what "no overflow is induced in the conversion" means in the above. What happens here:

```
duration<int, milli> ms(INT_MAX);
duration<int, micro> us = ms; // ???
```

An overflow is clearly induced in the conversion here: internally, we'll multiply `INT_MAX` by 1000. But that cannot be determined statically (in general), and so can't affect the result of overload resolution.

So what's actually supposed to happen? Are we actually just supposed to check that `Rep2` is no larger than `Rep`? (If so, what happens on overflow? Undefined behavior?)

It has been pointed out by Howard Hinnant:

This refers to the compile-time conversion factor to convert `Period2` to `Period`. If that conversion factor is not representable as a (reduced) `ratio<N, D>`, then the constructor is SFINAE'd out. This might happen (for example) converting years to picoseconds.

I would not have guessed that from the wording. Maybe replacing "no overflow is induced in the conversion" with "the result of `ratio_divide<Period2, Period>` is representable as a ratio" or similar would help?

[2018-06-18 after reflector discussion]

Priority set to 3

[2020-09-12 Jonathan adds a proposed resolution]

Since the result of the `ratio_divide` has to be a ratio, if it's not representable then the result simply isn't a valid type. Implementations are not required to make `ratio_divide` SFINAE-friendly to implement this constraint. They can perform the equivalent calculations to check if they would overflow, without actually using `ratio_divide`.

[2025-10-23; Reflector poll.]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 30.5.2 [\[time.duration.cons\]](#) as indicated:

```
template<class Rep2, class Period2>
constexpr duration(const duration<Rep2, Period2>& d);
```

-3- *Constraints:* [is_convertible_v<const Rep2&, rep>](#) is true. [ratio_divide<typename Period2::type, _period>](#) is a valid ratio specialization. [Either:](#)

- [treat_as_floating_point_v<rep>](#) is true; or

- o `ratio_divide<Period2, period>::den is 1 and treat_as_floating_point v<Rep2> is false.`

~~No overflow is induced in the conversion and `treat_as_floating_point v<rep>` is true or both `ratio_divide<Period2, period>::den is 1 and treat_as_floating_point v<Rep2>` is false.~~ [Note: This requirement prevents implicit truncation errors when converting between integral-based duration types. Such a construction could easily lead to confusion about the value of the duration. — end note]

3627. Inconsistent specifications for `std::make_optional` overloads

Section: 22.5.10 [\[optional.specalg\]](#) Status: [Voting](#) Submitter: Jiang An Opened: 2021-10-23 Last modified: 2025-10-30

Priority: 3

Discussion:

Three `std::make_optional` overloads are specified in 22.5.10 [\[optional.specalg\]](#). The first one is specified by "Returns:" and the other two are specified by "Effects: Equivalent to:". According to 16.3.2.4 [\[structure.specifications\]](#)/4, such uses of "Effects: Equivalent to:" propagate the *Constraints* specified for constructors. As the selected constructor for the first overload has "Constraints:" (22.5.3.2 [\[optional.ctor\]](#)/22), it seems that inconsistency is introduced here.

Existing implementations are inconsistent: libstdc++ constrains all three overloads, while libc++ and MSVC STL do not constrain any of them.

IMO all three overloads should be constrained.

[2022-01-29; Reflector poll]

Set priority to 3 after reflector poll.

[2025-10-16; Status changed: New → Tentatively Ready.]

Reflector poll in 2024-07 with eight supporting votes.

Proposed resolution:

This wording is relative to [N4901](#).

1. Modify 22.5.10 [\[optional.specalg\]](#) as indicated:

```
template<class T> constexpr optional<decay_t<T>> make_optional(T&& v);  
  
-3- ReturnsEffects: Equivalent to: return optional<decay_t<T>>(std::forward<T>(v));
```

4020. `extents::index-cast` weirdness

Section: 23.7.3.3.2 [\[mdspan.extents.expo\]](#) Status: [Voting](#) Submitter: Casey Carter Opened: 2023-11-29 Last modified: 2025-10-30

Priority: Not Prioritized

Discussion:

The exposition-only static member `index-cast` of `extents` is specified as (23.7.3.3.2 [\[mdspan.extents.expo\]](#)/9):

```
template<class OtherIndexType>  
static constexpr auto index-cast(OtherIndexType&& i) noexcept;
```

-9- Effects:

(9.1) — If `OtherIndexType` is an integral type other than `bool`, then equivalent to `return i;`,

(9.2) — otherwise, equivalent to `return static_cast<index_type>(i);`.

[Note 1: This function will always return an integral type other than `bool`. Since this function's call sites are constrained on convertibility of `OtherIndexType` to `index_type`, integer-class types can use the `static_cast` branch without loss of precision. — end note]

This function returns `T` when passed an rvalue of *cv*-unqualified integral type `T`, but `index_type` when passed a *cv*-qualified and/or lvalue argument of any integral type. It would seem more consistent and easier to reason about if 9.1 was instead conditional on `remove_cvref_t<OtherIndexType>`.

[2025-10-17; Reflector poll.]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

"Doesn't matter in this case, but logically `decay_t` seems like a better fit."

Proposed resolution:

This wording is relative to [N4964](#).

1. Modify 23.7.3.3.2 [\[mdspan.extents.expo\]](#) as indicated:

```
template<class OtherIndexType>
static constexpr auto index_cast(OtherIndexType&& i) noexcept;
```

-9- *Effects*:

(9.1) — If `remove_cvref_t<OtherIndexType>` is an integral type other than `bool`, then equivalent to `return i;`,

(9.2) — otherwise, equivalent to `return static_cast<index_type>(i);`.

[*Note 1*: This function will always return an integral type other than `bool`. Since this function's call sites are constrained on convertibility of `OtherIndexType` to `index_type`, integer-class types can use the `static_cast` branch without loss of precision.
— *end note*]

4136. Specify behavior of [linalg] Hermitian algorithms on diagonal with nonzero imaginary part

Section: 29.9.3 [linalg.general] **Status:** Voting **Submitter:** Mark Hoemmen **Opened:** 2024-08-09 **Last modified:** 2025-10-30

Priority: Not Prioritized

Discussion:

Addresses [US 170-278](#)

Mathematically, the diagonal elements of a Hermitian matrix must all have zero imaginary part (if they are complex numbers). 29.9.3 [linalg.general] paragraphs 3 and 4 govern the behavior of [linalg] functions that operate on "symmetric," "Hermitian," or "triangular" matrices. All such functions only access the specified triangle (lower or upper) of the matrix. Whatever is in the other triangle of the matrix doesn't matter; it's not even accessed. That gives well-defined behavior for "symmetric" and "triangular" matrices. However, both triangles include the diagonal. What should the "Hermitian" functions do if they encounter a diagonal element with nonzero imaginary part?

The current wording says that both the operation performed and the matrix itself are Hermitian, but does not clarify what happens if the latter is not true. For example, 29.9.14.3 [linalg.algs.blas2.hemv] says that `hermitian_matrix_vector_product` performs a

Hermitian matrix-vector product, taking into account the `Triangle` parameter that applies to the Hermitian matrix `A` (29.9.3 [linalg.general]).

Language like this appears in the specifications of all the functions whose names start with `hermitian`. The implication is that if the diagonal has an element with nonzero imaginary part, then the matrix is not Hermitian and therefore a precondition of the function has been violated. The result is undefined behavior.

We can get rid of this undefined behavior by defining what happens in this case. It turns out that Chapter 2 of the BLAS Standard already does this: It says that the Hermitian algorithms do not access the imaginary parts of diagonal elements. The reference Fortran BLAS implementations of CHEMV (single-precision Complex HERmitian Matrix-Vector product) and ZHEMV (double-precision complex HERmitian Matrix-Vector product) follow the BLAS Standard. CHEMV uses `real(A(j, j))` and ZHEMV uses `dbble(A(j, j))`, which means that the BLAS routines only access the real part of each diagonal element.

The clear design intent of [P1673R13](#) was to imitate the behavior of the BLAS Standard and reference BLAS unless otherwise specified. Thus, we propose to specify this behavior in the wording; we do not think this requires LEWG re-review.

In my view, it's fine to retain the existing wording like that in 29.9.14.3 [linalg.algs.blas2.hemv], that refers to a "Hermitian matrix". The matrix defined by the revised [linalg.general] instructions is unambiguously a Hermitian matrix. Just like the "other triangle" of a symmetric or triangular matrix does not affect the behavior of [linalg] symmetric resp. triangular algorithms, the wording fix here will ensure that any imaginary parts of diagonal elements will not affect the behavior of [linalg] Hermitian algorithms.

[2025-10-23; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4988](#).

1. Modify 29.9.3 [linalg.general] as indicated:

-4. For any function `F` that takes a parameter named `t`, `t` applies to accesses done through the parameter preceding `t` in the parameter list of `F`. Let `m` be such an access-modified function parameter. `F` will only access the triangle of `m` specified by `t`. **For accesses of diagonal elements `m[i, i]`, `F` will use the value `real-if-needed(m[i, i])` if the name of `F` starts with `hermitian`.** For accesses `m[i, j]` outside the triangle specified by `t`, `F` will use the value [...]

4166. `concat_view::end()` should be more constrained in order to support noncopyable iterators

Section: 25.7.18.2 [range.concat.view] **Status:** Voting **Submitter:** Yaito Kakeyama & Nana Sakisaka **Opened:** 2024-10-13 **Last modified:** 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [range.concat.view].

View all other [issues](#) in [range.concat.view].

Discussion:

There is a case that `concat(a, b)` compiles but `concat(b, a)` does not.

```
auto range_copyable_it = std::vector<int>{1, 2, 3};

std::stringstream ss{"4 5 6"};
auto range_noncopyable_it = std::views::istream<int>(ss);

auto view1 = std::views::concat(range_copyable_it, range_noncopyable_it);
static_assert(std::ranges::range<decltype(view1)>); // ok
assert(std::ranges::equal(view1, std::vector{1, 2, 3, 4, 5, 6})); // ok

auto view2 = std::views::concat(range_noncopyable_it, range_copyable_it);
// static_assert(std::ranges::range<decltype(view2)>); // error
// assert(std::ranges::equal(view2, std::vector{4, 5, 6, 1, 2, 3})); // error
```

The reason behind this is as follows:

Firstly, if all `Views...` satisfy the `std::ranges::range` concept, then `concat_view` should also satisfy it. However, if any of the `Views...` have a noncopyable iterator and the last view is `common_range`, the current `concat_view` fails to model a range.

For `concat_view` to model a range, its sentinel must satisfy `std::semiregular`, but `concat_view::end()` returns a `concat_view::iterator`, which is noncopyable if the underlying iterator is noncopyable. This issue arises from the proposed implementation where the iterator uses `std::variant`. Although this specification is exposition-only, even if an alternative type-erasure mechanism is used, copying is still required if the user attempts to copy an iterator.

To resolve the issue, `concat_view::end()` can and should fallback to returning `std::default_sentinel` in such cases.

Unfortunately, as a side effect, this fix would prevent `concat_view` from being a `common_range` in certain situations. According to [P2542R8](#):

concat_view can be common_range if the last underlying range models common_range

However, this is no longer true after applying our fix. That said, these two issues cannot be resolved simultaneously due to implementability. Therefore, we suggest applying our fix regardless and accepting that `concat_view` will not always inherit `common_range`. Note that the current draft ([N4988](#)) does not explicitly specify when `concat_view` can model `common_range`, so no addition is required for mentioning this point.

A similar issue had been reported as [3385](#), which was eventually adopted as a C++20 DR. This DR indicates that LWG approved the decision to require copyable in order to model a `common_iterator`.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4993](#).

1. Modify 25.7.18.2 [\[range.concat.view\]](#) as indicated:

```
constexpr auto end() const
requires (range<const Views> && ...) && concatable<const Views...>;
```

-7- *Effects*: Let *is-const* be true for the const-qualified overload, and false otherwise. Equivalent to:

```
constexpr auto N = sizeof...(Views);
if constexpr ((semiregular<iterator t<maybe-const<is-const, Views>>> && ...) &&
               common_range<maybe-const<is-const, Views...[N - 1]>>>) {
    return iterator<is-const>(this, in_place_index<N - 1>,
                               ranges::end(std::get<N - 1>(views_)));
} else {
    return default_sentinel;
}
```

[2025-03-05; Hewill Kang provides improved wording]

[2025-10-23; Reflector poll.]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5001](#).

1. Modify 25.7.18.2 [\[range.concat.view\]](#) as indicated:

```
constexpr auto end() const
requires (range<const Views> && ...) && concatable<const Views...>;
```

-7- *Effects*: Let *is-const* be true for the const-qualified overload, and false otherwise. Equivalent to:

```
constexpr auto N = sizeof...(Views);
if constexpr (all-forward<is-const, Views...> &&
               common_range<maybe-const<is-const, Views...[N - 1]>>>) {
    return iterator<is-const>(this, in_place_index<N - 1>,
                               ranges::end(std::get<N - 1>(views_)));
} else {
    return default_sentinel;
}
```

4253. basic_const_iterator should provide iterator_type

Section: 24.5.3.3 [\[const.iterators.iterator\]](#) Status: [Voting](#) Submitter: Hewill Kang Opened: 2025-04-29 Last modified: 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [\[const.iterators.iterator\]](#).

View all other [issues](#) in [\[const.iterators.iterator\]](#).

Discussion:

Currently, iterator adaptors in `<iterator>` that wrap a single iterator such as `reverse_iterator`, `move_iterator`, and `counted_iterator` all provide a public `iterator_type` member for users to access the underlying iterator type, except for `basic_const_iterator` ([demo](#)):

```
#include <iterator>

using I = int*;
using RI = std::reverse_iterator<I>;
using MI = std::move_iterator<I>;
using CI = std::counted_iterator<I>;
using BI = std::basic_const_iterator<I>;

static_assert(std::same_as<RI::iterator_type, I>);
static_assert(std::same_as<MI::iterator_type, I>);
static_assert(std::same_as<CI::iterator_type, I>);
static_assert(std::same_as<BI::iterator_type, I>); // error
```

It seems reasonable to add one for `basic_const_iterator` for consistency.

[2025-06-12; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 24.5.3.3 [\[const.iterators.iterator\]](#) as indicated:

```
namespace std {
[...]
template<input_iterator Iterator>
class basic_const_iterator {
    Iterator current_ = Iterator();           // exposition only
    using reference = iter_const_reference_t<Iterator>; // exposition only
    using rvalue-reference = // exposition only
        iter_const-rvalue-reference-t<Iterator>;

public:
    using iterator_type = Iterator;
    using iterator_concept = see below;
    using iterator_category = see below; // not always present
    using value_type = iter_value_t<Iterator>;
    using difference_type = iter_difference_t<Iterator>;
    [...]
};
}
```

4255. move_only_function constructor should recognize empty copyable_functions

Section: 22.10.17.4.3 [\[func.wrap.move.ctor\]](#) Status: [Voting](#) Submitter: Tomasz Kamiński Opened: 2025-05-12 Last modified: 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [\[func.wrap.move.ctor\]](#).

View all other [issues](#) in [\[func.wrap.move.ctor\]](#).

Discussion:

The standard currently requires that constructing `move_only_function` from empty `copyable_function`, creates a non-empty `move_only_function`, that contains an empty `copyable_function` as the target. For example:

```
std::copyable_function<int(int)> ce;
std::move_only_function<int(int)> me(ce);
```

We require that invoking `me(1)` is undefined behavior (as it leads to call to the `ce(1)`), however it cannot be detected in the user code, as `me != nullptr` is true.

We should require the `move_only_function(F&& f)` constructor to create an empty object, if `f` is an instantiation of `copyable_function` and `f == nullptr` is true, i.e. `f` does not contain target object.

This simplifies implementing avoidance of double wrapping per 22.10.17.1 [\[func.wrap.general\]](#) p2, as transferring the target produces an empty functor.

The `copyable_function` cannot be constructed from `move_only_function`, as it requires functor to be copyable. Invoking an empty `std::function` has well defined behavior (throws `bad_function_call`), and wrapping such object into other functors should reproduce that behavior.

[2025-10-22; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 22.10.17.4.3 [\[func.wrap.move.ctor\]](#) as indicated:

```
template<class F> move_only_function(F&& f);
```

[...]

-8- *Postconditions*:: *this has no target object if any of the following hold:

(8.1) — `f` is a null function pointer value, ~~or~~

(8.2) — `f` is a null member pointer value, or

(8.2) — `remove_cvref_t<F>` is a specialization of the `move_only_function` [or copyable_function](#) class template, and `f` has no target object.

[4256](#). Incorrect constrains for `function_ref` constructors from `nontype_t`

Section: 22.10.17.6.3 [\[func.wrap.ref.ctor\]](#) **Status:** [Voting](#) **Submitter:** Tomasz Kamiński **Opened:** 2025-05-14 **Last modified:** 2025-10-30

Priority: Not Prioritized

Discussion:

For the following class:

```
struct M
{
    void operator()();
};
```

The constructor of `function_ref<void()>` from `nontype_t` is considered to be valid candidate (`is_constructible_v<function_ref<void()>, nontype_t<M{}>>` is true), despite the fact that the corresponding invocation of template argument object, that is `const lvalue`, is ill-formed. As consequence we produce a hard error from inside of this constructor.

This is caused by the fact that for constructors with non-type `auto f` parameter, we are checking if `is_invocable_using<F>` is true, where `F` is `decltype(f)` i.e. `M` for the example. We should use `const F&` or `decltype(f)`.

[2025-10-21; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 22.10.17.6.3 [\[func.wrap.ref.ctor\]](#) as indicated:

```
template<auto f> constexpr function_ref(nontype_t<f>) noexcept;
```

-8- Let `F` be `decltype(f)`.

-9- *Constraints*: `is_invocable_using<const F&>` is true.

[...]

```
template<auto f, class U> constexpr function_ref(nontype_t<f>, U&& obj) noexcept;
```

-12- Let `T` be `remove_reference_t<U>` and `F` be `decltype(f)`.

-13- *Constraints*::

(13.1) — `is_rvalue_reference_v<U&&>` is false, and

(13.2) — `is_invocable_using<const F&, T cv&>` is true.

[...]

```
template<auto f, class T> constexpr function_ref(nontype_t<f>, T cv* obj) noexcept;
```

-17- Let F be decltype(f).

-16- Constraints: *is-invocable-using*<const F&, T cv*> is true.

[...]

[4257](#). Stream insertion for `chrono::local_time` should be constrained

Section: 30.7.9 [\[time.clock.local\]](#) **Status:** [Voting](#) **Submitter:** Jonathan Wakely **Opened:** 2025-05-16 **Last modified:** 2025-10-30

Priority: Not Prioritized

Discussion:

Stream insertion for `chrono::local_time` is defined in terms of conversion to `chrono::sys_time`, but not all `chrono::sys_time` specializations can be inserted into an ostream, because one of the overloads is constrained and the other requires convertibility to `chrono::sys_days` (see 30.7.2.3 [\[time.clock.system.nonmembers\]](#)).

This means the following code fails to compile:

```
#include <iostream>
#include <chrono>

template<typename T>
concept ostream_insertable = requires (std::ostream& o, const T& t) { o << t; };

using D = std::chrono::duration<double>;

int main() {
    if constexpr (ostream_insertable<std::chrono::sys_time<D>>)
        std::cout << std::chrono::sys_time<D>{};
    if constexpr (ostream_insertable<std::chrono::local_time<D>>)
        std::cout << std::chrono::local_time<D>{}; // FAIL
}
```

The first condition is false, because there's no overload that's suitable. The second is true, because the operator<< overload for `chrono::local_time` isn't constrained and so insertion appears to be valid. But actually trying to use it is ill-formed, because it tries to convert the `local_time<D>` to a `sys_time<D>` and then insert that, which isn't valid.

[2025-08-21; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 30.7.9 [\[time.clock.local\]](#) as indicated:

```
template<class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const local_time<Duration>& lt);
```

-?- Constraints: `os << sys_time<Duration>{lt.time_since_epoch()}` is a valid expression.

-2- Effects:

`os << sys_time<Duration>{lt.time_since_epoch()};`

-3- Returns: `os`.

[4265](#). `std::midpoint` should not accept `const bool`

Section: 26.10.16 [\[numeric.ops.midpoint\]](#) **Status:** [Voting](#) **Submitter:** Jan Schultke **Opened:** 2025-05-21 **Last modified:** 2025-10-30

Priority: Not Prioritized

View all other [issues](#) in `[numeric.ops.midpoint]`.

Discussion:

The constraints of the first overload of `std::midpoint` are as follows:

```
template<class T>
constexpr T midpoint(T a, T b) noexcept;
```

-1- *Constraints*: T is an arithmetic type other than `bool`.

It does not appear intentional that `const bool` is supported considering that 26.10.14 [numeric.ops.gcd] excludes *cv* `bool`.

More generally, it is questionable that *cv*-qualified arithmetic types aren't excluded in general, considering that 26.10.17 [numeric.sat] excludes them with the "signed or unsigned integer type" constraint. However, tightening the constraints for these other functions seems evolutionary, not like fixing an obvious oversight, and so it likely requires a separate issue.

[2025-10-21; Reflector poll.]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

This requires template argument to be explicitly specified, i.e. `midpoint<const bool>`. I would prefer to address all *cv*-qualified types at once, e.g. *Constraints*: `remove_cv_t<T>` is an arithmetic type other than `bool`."

"This is locally consistent with `gcd` and `lcm` which only exclude *cv* `bool`. [algorithms.requirement] p15 makes it unspecified to use an explicit template argument list here, so `midpoint<const bool>` and `midpoint<const int>` are already unspecified, this issue just ensures that `const bool` is explicitly rejected, like `bool`."

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 26.10.16 [numeric.ops.midpoint] as indicated:

```
template<class T>
constexpr T midpoint(T a, T b) noexcept;

-1- Constraints: T is an arithmetic type other than cv bool.
```

4266. layout_stride::mapping should treat empty mappings as exhaustive

Section: 23.7.3.4.7 [mdspan.layout.stride] **Status:** [Voting](#) **Submitter:** Tomasz Kamiński **Opened:** 2025-05-22 **Last modified:** 2025-10-30

Priority: Not Prioritized

View all other [issues](#) in [mdspan.layout.stride].

Discussion:

Mapping over an empty multidimensional index space is always exhaustive according to the corresponding definitions from 23.7.3.4.2 [mdspan.layout.reqmts] p16.

However, the current specification of `layout_stride::mapping` does not consider whether some of the empty multidimensional index spaces are unique or exhaustive. For illustration, the mapping with the following configuration is not considered exhaustive according to the current specification of 23.7.3.4.7.4 [mdspan.layout.stride.obs] bullet 5.2:

```
extents: 2, 2, 0
strides: 2, 6, 20
```

This prevents the implementation from implementing `sm.is_exhaustive()` as `sm.fwd-prod-of-extents(sm::extents_type::rank()) == sm.required_span_size()`. For all mappings with size greater than zero, such an expression provides an answer consistent with the standard. However, it always returns true for an empty mapping, such as shown in the example.

We should make such implementation conforming, and require `is_exhaustive()` to return true for empty mappings.

For consistency, we could update `is_always_exhaustive()` to recognize mapping with `rank() == 0`, and one for which at least one of the static extents is equal to zero (i.e., they always represent a multidimensional index space).

[2025-06-12; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 23.7.3.4.7.1 [mdspan.layout.stride.overview] as indicated:

```
namespace std {
template<class Extents>
class layout_stride::mapping {
    [...]
    static constexpr bool is_always_unique() noexcept { return true; }
    static constexpr bool is_always_exhaustive() noexcept { return false; }
    static constexpr bool is_always_strided() noexcept { return true; }
    [...]
}
```

```
};  
}
```

2. Modify 23.7.3.4.7.4 [\[mdspan.layout.stride.obs\]](#) as indicated:

[...]

```
static constexpr bool is_always_exhaustive() noexcept;
```

-?- Returns: true if *rank* is 0 or if there is a rank index *r* of *extents()* such that *extents_type::static_extent(r)* is 0, otherwise false.

```
constexpr bool is_exhaustive() const noexcept;
```

-5- Returns:

(5.1) — true if *rank_* or the size of the multidimensional index space *m.extents()* is 0.

(5.2) — [...]

(5.3) — [...]

4269. unique_copy passes arguments to its predicate backwards

Section: 26.7.9 [\[alg.unique\]](#) Status: [Voting](#) Submitter: Jonathan Wakely Opened: 2025-05-29 Last modified: 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [\[alg.unique\]](#).

View all other [issues](#) in [\[alg.unique\]](#).

Discussion:

For the unique algorithms, 26.7.9 [\[alg.unique\]](#) p1 says:

1. Let *pred* be `equal_to{}` for the overloads with no parameter *pred*, and let *E* be

(1.1) — `bool(pred(*(i - 1), *i))` for the overloads in namespace `std`;

(1.2) — `bool(invoker(comp, invoker(proj, *(i - 1)), invoker(proj, *i)))` for the overloads in namespace `ranges`.

However for the `unique_copy` algorithms, 26.7.9 [\[alg.unique\]](#) p6 says that the arguments **i* and **(i-1)* should be reversed:

6. Let *pred* be `equal_to{}` for the overloads with no parameter *pred*, and let *E* be

(6.1) — `bool(pred(*i, *(i - 1)))` for the overloads in namespace `std`;

(6.2) — `bool(invoker(comp, invoker(proj, *i), invoker(proj, *(i - 1))))` for the overloads in namespace `ranges`.

This reversed order is consistent with the documentation for [SGI STL unique_copy](#), although the docs for [SGI STL unique](#) show reversed arguments too, and the C++ standard doesn't match that.

A survey of known implementations shows that all three of `libstdc++`, `libc++`, and `MSVC STL` use the `pred(*(i - 1), *i)` order for all of `std::unique`, `std::unique_copy`, `ranges::unique`, and `ranges::unique_copy`. The `range-v3` library did the same, and even the `SGI STL` did too (despite what its docs said). Only two implementations were found which match the spec and use a different argument order for `unique` and `unique_copy`, Casey Carter's (`cmcstl2`) and Fraser Gordon's.

In the absence of any known rationale for `unique` and `unique_copy` to differ, it seems sensible to make `unique_copy` more consistent with `unique` (and with the majority of implementations stretching back three decades).

[2025-10-17; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Fixed misplaced `)` in the (6.1) change as pointed out on reflector, and rebased on [N5014](#).

"I remain unconvinced that this actually matters given the equivalence relation requirement."

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 26.7.9 [\[alg.unique\]](#) as indicated:

6. Let *pred* be `equal_to{}` for the overloads with no parameter *pred*, and let *E(i)* be

(6.1) — `bool(pred(*i, *(i - 1), *i))` for the overloads in namespace `std`;

(6.2) — `bool(invoker(comp, invoker(proj, *i), invoker(proj, *(i - 1)), invoker(proj, *i)))` for the overloads in namespace `ranges`.

4274. The chrono::hh_mm_ss constructor is ill-formed for unsigned durations

Section: 30.9.2 [\[time.hms.members\]](#) Status: [Voting](#) Submitter: Michael Welsh Duggan Opened: 2025-06-04 Last modified: 2025-10-30

Priority: Not Prioritized

Discussion:

In 30.9.2 [\[time.hms.members\]](#), paragraph 3, the current wording for the constructor of hh_mm_ss expresses some of its requirements in terms of `abs(d)`, which is assumed to be `chrono::abs(chrono::duration)`. `chrono::abs` is not defined, however, for durations with an unsigned representation. I believe that not being able to create hh_mm_ss objects from unsigned durations is unintentional.

Moreover, `is_constructible_v<hh_mm_ss<ud>, ud>` is required to be true by the standard for any duration, so making it actually work makes a lot of sense.

[2025-06-13; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 30.9.2 [\[time.hms.members\]](#) as indicated:

```
constexpr explicit hh_mm_ss(Duration d);
```

-3- *Effects*: Constructs an object of type hh_mm_ss which represents the Duration d with precision precision.

(3.1) — Initializes `is_neg` with `d < Duration::zero()`. **Let ABS D represent -d if is_neg is true and d otherwise.**

(3.2) — Initializes `h` with `duration_cast<chrono::hours>(abs(d)ABS D)`.

(3.3) — Initializes `m` with `duration_cast<chrono::minutes>(abs(d)ABS D - hours())`.

(3.4) — Initializes `s` with `duration_cast<chrono::seconds>(abs(d)ABS D - hours() - minutes())`.

(3.5) — If `treat_as_floating_point_v<precision::rep>` is true, initializes `ss` with `abs(d)ABS D - hours() - minutes() - seconds()`. Otherwise, initializes `ss` with `duration_cast<precision>(abs(d)ABS D - hours() - minutes() - seconds())`.

4275. std::dynamic_extent should also be defined in <mdspan>

Section: 23.7.3.2 [\[mdspan.syn\]](#) Status: [Voting](#) Submitter: Aiden Grossman Opened: 2025-06-06 Last modified: 2025-10-30

Priority: 3

View all other [issues](#) in `[mdspan.syn]`.

Discussion:

`std::dynamic_extent` can be used in certain circumstances in `std::mdspan`, such as with padded layouts. However, `std::dynamic_extent` is currently only defined in `<span>` which necessitates including `<span>` solely for the `std::dynamic_extent` definition.

Previous resolution [SUPERSEDED]:

This wording is relative to [N5008](#).

1. Modify 23.7.3.2 [\[mdspan.syn\]](#), header `<span>` synopsis, as indicated:

```
// all freestanding
namespace std {
    // constants
    inline constexpr size_t dynamic_extent = numeric_limits<size_t>::max();

    // 23.7.3.3 [mdspan.extents], class template extents
    template<class IndexType, size_t... Extents>
    class extents;

    [...]
}
```

[2025-06-10; Jonathan provides improved wording]

[2025-10-15; Reflector poll]

Set priority to 3 after reflector poll.

[2025-10-17; Reflector poll.]

Proposed resolution:

1. Modify 23.7.1 [\[views.general\]](#) as indicated:

-?- In addition to being available via inclusion of the `<span>` header, `dynamic_extent` is available when the header `<mdspan>` is included.

Section: 23.3.3.5 [\[array.zero\]](#) **Status:** [Voting](#) **Submitter:** Jan Schultke **Opened:** 2025-06-08 **Last modified:** 2025-10-30

View other [active issues](#) in [array.zero].

View all other [issues](#) in [array.zero].

Without this paragraph, `front()` and `back()` would be hardened as well, as specified in 23.2.4 [\[sequence.reqmts\]](#).

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 23.3.3.5 [\[array.zero\]](#) as indicated:

~~3- The effect of calling `front()` or `back()` for a zero-sized array is undefined.~~

Section: 29.10.8.7 [[simd.loadstore](#)] **Status:** [Voting](#) **Submitter:** Tim Song **Opened:** 2025-06-21 **Last modified:** 2025-10-30

View other [active issues](#) in [simd.loadstore].

View all other [issues](#) in [simd.loadstore].

The *Effects*: element of `std::simd::partial_load` (after the latest rename) uses `T` but that is not defined anywhere. It should be `V::value_type`.

Also, this paragraph should be a *Returns:* element.

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 29.10.8.7 [\[simd.loadstore\]](#) as indicated:

```
template<class V = see below, ranges::contiguous_range R, class... Flags>
    requires ranges::sized_range<R>
    constexpr V partial_load(R&& r, flags<Flags...> f = {});
[...]
```

```
template<class V = see below, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
    constexpr V partial_load(I first, S last, const typename V::mask_type& mask,
        flags<Flags...> f = {});
```

- 6- [...]
- 7- Mandates: [...]
- 8- Preconditions: [...]
- 9- ~~Effects: Initializes the~~**Returns: A basic simd object whose** i^{th} element **is initialized** with `mask[i]` && `i < ranges::size(r) ? static_cast<T>(ranges::data(r)[i]) : T()` for all `i` in the range of `[0, V::size())`, **where T is V::value_type**.
- 10- Remarks: The default argument for template parameter V is `basic_simd<ranges::range_value_t<R>>`.

4286. Some more feature-test macros for fully freestanding features are not marked freestanding

Section: 17.3.2 [\[version.syn\]](#), 20.2.2 [\[memory.syn\]](#) Status: [Voting](#) Submitter: Yihe Li Opened: 2025-06-17 Last modified: 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [\[version.syn\]](#).

View all other [issues](#) in [\[version.syn\]](#).

Discussion:

[P1642R11](#) (accepted in C++23) plus LWG [4189^{\(1\)}](#) (accepted in Hagenberg) added nearly the entire `<ranges>` header to freestanding. However, the only feature-test macro being added to freestanding is `__cpp_lib_ranges_cache_latest`, which seems weird, since `views::enumerate` is also added to freestanding following the blanket comment strategy, but its feature-test macro remains not in freestanding. In retrospective, since all range algorithms are in freestanding via [P2976](#), all `__cpp_lib_ranges_*` FTMs (except `__cpp_lib_ranges_generate_random` since `ranges::generate_random` is not in freestanding) should probably be marked as freestanding.

Furthermore, LWG [4126^{\(1\)}](#) left out some other FTMs for fully freestanding features. They are also added in the following wording.

A note about `is_sufficiently_aligned`: [P2897R7](#) does indicate in 5.7.6.1 that the function should be freestanding, but somehow the wording didn't say so. The following wording includes the function and its FTM anyway since hopefully this is just an omission when wording the paper.

[2025-10-14; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 17.3.2 [\[version.syn\]](#), header `<version>` synopsis, as indicated:

```
[...]
#define __cpp_lib_aligned_accessor          202411L // freestanding, also in <mdspan>
[...]
#define __cpp_lib_array_constexpr          201811L // freestanding, also in <iterator>, <array>
[...]
#define __cpp_lib_clamp                    201603L // freestanding, also in <algorithm>
[...]
#define __cpp_lib_constexpr_numeric        201911L // freestanding, also in <numeric>
[...]
#define __cpp_lib_function_ref              202306L // freestanding, also in <functional>
#define __cpp_lib_gcd_lcm                   201606L // freestanding, also in <numeric>
[...]
#define __cpp_lib_integer_comparison_functions 202002L // freestanding, also in <utility>
[...]
#define __cpp_lib_is_sufficiently_aligned    202411L // freestanding, also in <memory>
[...]
#define __cpp_lib_ranges_contains           202207L // freestanding, also in <algorithm>
#define __cpp_lib_ranges_enumerate          202302L // freestanding, also in <ranges>
#define __cpp_lib_ranges_find_last          202207L // freestanding, also in <algorithm>
#define __cpp_lib_ranges_fold               202207L // freestanding, also in <algorithm>
[...]
#define __cpp_lib_ranges_iota               202202L // freestanding, also in <numeric>
[...]
#define __cpp_lib_ranges_starts_ends_with   202106L // freestanding, also in <algorithm>
[...]
#define __cpp_lib_robust_nonmodifying_seq_ops 201304L // freestanding, also in <algorithm>
#define __cpp_lib_sample                    201603L // freestanding, also in <algorithm>
#define __cpp_lib_saturation_arithmetic     202311L // freestanding, also in <numeric>
[...]
```

2. Modify 20.2.2 [\[memory.syn\]](#), header `<memory>` synopsis, as indicated:

```
[...]
template<size_t Alignment, class T>
    bool is_sufficiently_aligned(T* ptr);    // freestanding
[...]
```

4291. explicit map(const Allocator&) should be constexpr

Section: 23.4.3.1 [map.overview] Status: [Voting](#) Submitter: Jonathan Wakely Opened: 2025-07-10 Last modified: 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [map.overview].

View all other [issues](#) in [map.overview].

Discussion:

The intent of [P3372R3](#) was for all container constructors to be constexpr, but during application of the paper to the working draft it was observed that one map constructor was missed.

[2025-08-21; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 23.4.3.1 [map.overview] as indicated:

```
// 23.4.3.2, construct/copy/destroy
constexpr map() : map(Compare()) { }
constexpr explicit map(const Compare& comp, const Allocator& = Allocator());
template<class InputIterator>
constexpr map(InputIterator first, InputIterator last,
               const Compare& comp = Compare(), const Allocator& = Allocator());
template<container-compatible-range <value_type> R>
constexpr map(from_range_t, R&& rg, const Compare& comp = Compare(),
               const Allocator& = Allocator());
constexpr map(const map& x);
constexpr map(map&& x);
constexpr explicit map(const Allocator&);
constexpr map(const map&, const type_identity_t<Allocator>&);
constexpr map(map&&, const type_identity_t<Allocator>&);
constexpr map(initializer_list<value_type>, const Compare& = Compare(),
               const Allocator& = Allocator());
```

4292. Unordered container local iterators should be constexpr iterators

Section: 23.5.3.1 [unord.map.overview], 23.5.4.1 [unord.multimap.overview], 23.5.6.1 [unord.set.overview], 23.5.7.1 [unord.multiset.overview] Status: [Voting](#)
Submitter: Jonathan Wakely Opened: 2025-07-10 Last modified: 2025-10-30

Priority: Not Prioritized

Discussion:

The intent of [P3372R3](#) was for all container iterators to be constexpr iterators, but during application of the paper to the working draft it was observed that unordered containers don't say it for their local iterators.

[2025-08-29; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 23.5.3.1 [unord.map.overview] as indicated:

-4- The types iterator, ~~and~~ const_iterator, **local iterator, and const local iterator** meet the constexpr iterator requirements (24.3.1 [iterator.requirements.general]).

2. Modify 23.5.4.1 [unord.multimap.overview] as indicated:

-4- The types iterator, ~~and~~ const_iterator, **local iterator, and const local iterator** meet the constexpr iterator requirements (24.3.1 [iterator.requirements.general]).

3. Modify 23.5.6.1 [unord.set.overview] as indicated:

-4- The types iterator, ~~and~~ const_iterator, **local iterator, and const local iterator** meet the constexpr iterator requirements (24.3.1 [iterator.requirements.general]).

4. Modify 23.5.7.1 [unord.multiset.overview] as indicated:

-4- The types `iterator`, `const_iterator`, `local_iterator`, and `const_local_iterator` meet the `constexpr` iterator requirements (24.3.1 [\[iterator.requirements.general\]](#)).

4293. `span::subspan/first/last` chooses wrong constructor when `T` is `const-qualified bool`

Section: 23.7.2.2.4 [\[span.sub\]](#) Status: [Voting](#) Submitter: Yuhan Liu Opened: 2025-07-11 Last modified: 2025-10-30

Priority: Not Prioritized

View all other [issues](#) in `[span.sub]`.

Discussion:

In section 23.7.2.2.4 [\[span.sub\]](#), paragraphs p12, p14, and p16 erroneously use the initializer list constructor for `span` instead of the intended `iterator/count` constructor.

Specifically, in these paragraphs, the standard states:

Effects: Equivalent to: `return {data(), count};`

or some variant of `return {pointer, size}`. As reported in [GCC bug 120997](#) this results in a `span` that points to invalid stack memory. This can be reproduced on GCC 15.1 for `subspan`, `first`, and `last`: <https://godbolt.org/z/r9nrdWscq>.

A proposed fix (thanks to Jonathan Wakely) could look like this following:

```
return span<element_type>(data(), count);
```

for the affected paragraphs, which would explicitly specify the constructor used.

[2025-07-11; Jonathan adds proposed resolution]

The meaning of those *Effects*: paragraphs was changed for C++26 by [P2447R6](#) which added the `span(initializer_list)` constructor. A simpler demo is:

```
bool a[5]{};
std::span<const bool> s(a);
std::span<const bool> s2 = s.first(5);
assert(s2.size() == 5); // OK in C++23, fails in C++26
assert(s2.data() == a); // OK in C++23, fails in C++26
```

The proposed resolution is to use `R(data(), count)` instead of `{data(), count}`. The former always (uniformly) means the same thing, but for the latter the meaning of list-initialization depends on the types. The list-initialization form will choose the initializer-list constructor when `data()` and `count` are both convertible to the element type.

[2025-08-21; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 23.7.2.2.4 [\[span.sub\]](#) as indicated:

```
template<size_t Count> constexpr span<element_type, Count> first() const;
```

-1- *Mandates:* `Count <= Extent` is true.

-2- *Hardened preconditions:* `Count <= size()` is true.

-3- *Effects:* Equivalent to: `return R({data(), Count});` where `R` is the return type.

```
template<size_t Count> constexpr span<element_type, Count> last() const;
```

-4- *Mandates:* `Count <= Extent` is true.

-5- *Hardened preconditions:* `Count <= size()` is true.

-6- *Effects:* Equivalent to: `return R({data() + (size() - Count), Count});` where `R` is the return type.

```
template<size_t Offset, size_t Count = dynamic_extent>
constexpr span<element_type, see below> subspan() const;
```

-7- *Mandates:*

`Offset <= Extent && (Count == dynamic_extent || Count <= Extent - Offset)`

is true.

-8- *Hardened preconditions*:

```
Offset <= size() && (Count == dynamic_extent || Count <= size() - Offset)
```

is true.

-9- *Effects*: Equivalent to:

```
return span<ElementType, see below>(
    data() + Offset, Count != dynamic_extent ? Count : size() - Offset);
```

-10- *Remarks*: The second template argument of the returned span type is:

```
Count != dynamic_extent ? Count
    : (Extent != dynamic_extent ? Extent - Offset
    : dynamic_extent)
```

```
constexpr span<element_type, dynamic_extent> first(size_type count) const;
```

-11- *Hardened preconditions*: count <= size() is true.

-12- *Effects*: Equivalent to: return `R(data(), count);`; *where R is the return type.*

```
constexpr span<element_type, dynamic_extent> last(size_type count) const;
```

-13- *Hardened preconditions*: count <= size() is true.

-14- *Effects*: Equivalent to: return `R(data() + (size() - count), count);`; *where R is the return type.*

```
constexpr span<element_type, dynamic_extent> subspan(
    size_type offset, size_type count = dynamic_extent) const;
```

-15- *Hardened preconditions*:

```
offset <= size() && (count == dynamic_extent || count <= size() - offset)
```

is true.

-16- *Effects*: Equivalent to:

```
return R(data() + offset, count == dynamic_extent ? size() - offset : count);
```

where R is the return type.

4294. `bitset(const CharT*)` constructor needs to be constrained

Section: 22.9.2.2 [[bitset.cons](#)] **Status:** [Voting](#) **Submitter:** Jonathan Wakely **Opened:** 2025-07-12 **Last modified:** 2025-10-30

Priority: Not Prioritized

View all other [issues](#) in [[bitset.cons](#)].

Discussion:

This code might be ill-formed, with an error outside the immediate context that users cannot prevent:

```
#include <bitset>
struct NonTrivial { ~NonTrivial() { } };
static_assert( ! std::is_constructible_v<std::bitset<1>, NonTrivial*> );
```

The problem is that the `bitset(const CharT*)` constructor tries to instantiate `basic_string_view<NonTrivial>` to find its `size_type`, and that instantiation might ill-formed, e.g. if `std::basic_string_view` or `std::char_traits` has a static assert enforcing the requirement for their character type to be sufficiently char-like. 27.1 [[strings.general](#)] defines a char-like type as "any non-array trivially copyable standard-layout (6.9.1 [[basic.types.general](#)]) type T where `is_trivially_default_constructible_v<T>` is true."

[2025-08-21; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5008](#).

1. Modify 22.9.2.2 [[bitset.cons](#)] as indicated:

```
template<class charT>
constexpr explicit bitset(
```

```
const charT* str,
typename basic_string_view<charT>::size_type n = basic_string_view<charT>::npos,
charT zero = charT('0'),
charT one = charT('1'));
```

-?- Constraints:

- `is_array v<charT>` is false.
- `is_trivially_copyable v<charT>` is true.
- `is_standard_layout v<charT>` is true, and
- `is_trivially_default_constructible v<charT>` is true.

-8- *Effects:* As if by:

```
bitset(n == basic_string_view<charT>::npos
? basic_string_view<charT>(str)
: basic_string_view<charT>(str, n),
0, n, zero, one)
```

4297. Missing permutable constraint for iterator overloads in Parallel Range Algorithms

Section: 26.4 [\[algorithm.syn\]](#), 26.7.8 [\[alg.remove\]](#), 26.8.5 [\[alg.partitions\]](#) Status: [Voting](#) Submitter: Ruslan Arutyunyan Opened: 2025-06-27 Last modified: 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [\[algorithm.syn\]](#).

View all other [issues](#) in [\[algorithm.syn\]](#).

Discussion:

The [P3179R9](#): Parallel Range Algorithms paper was accepted to C++ working draft for C++ 26. Unfortunately, there is an oversight for three algorithms — `remove`, `remove_if` and `partition` — where the permutable constraint is missing. This applies to "Iterator and Sentinel" overloads only. The issue exists in 26.4 [\[algorithm.syn\]](#) as well as in per-algorithm sections: 26.8.5 [\[alg.partitions\]](#) and 26.7.8 [\[alg.remove\]](#).

[2025-10-21; Reflector poll.]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 26.4 [\[algorithm.syn\]](#), header `<algorithm>`, as indicated:

```
[...]
template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, class T = projected_value_t<I, Proj>>
    requires permutable<I> &&
        indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
    subrange<I> remove(Ep& exec, I first, S last, const T& value,
        Proj proj = {}); // freestanding-deleted
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        class T = projected_value_t<iterator_t<R>, Proj>>
    requires permutable<iterator_t<R>> &&
        indirect_binary_predicate<ranges::equal_to,
            projected<iterator_t<R>, Proj>, const T*>
    borrowed_subrange_t<R>
    remove(Ep&& exec, R&& r, const T& value, Proj proj = {}); // freestanding-deleted
[...]
```

```
template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
    requires permutable<I>
    subrange<I>
    remove_if(Ep& exec, I first, S last, Pred pred, Proj proj = {}); // freestanding-deleted
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    requires permutable<iterator_t<R>>
    borrowed_subrange_t<R>
    remove_if(Ep& exec, R& r, Pred pred, Proj proj = {}); // freestanding-deleted
[...]
```

```
template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
    requires permutable<I>
    subrange<I>
    partition(Ep&& exec, I first, S last, Pred pred, Proj proj = {}); // freestanding-deleted
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    requires permutable<iterator_t<R>>
    borrowed_subrange_t<R>
    partition(Ep&& exec, R&& r, Pred pred, Proj proj = {}); // freestanding-deleted
[...]
```

2. Modify 26.7.8 [\[alg.remove\]](#) as indicated:

```
[...]
template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, class T = projected_value_t<I, Proj>>
    requires permutable<I> &&
        indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
    subrange<I>
        ranges::remove(Ep& exec, I first, S last, const T& value, Proj proj = {});
template<execution-policy Ep, sized_random_access_range R, class Proj = identity,
        class T = projected_value_t<iterator_t<R>, Proj>>
    requires permutable<iterator_t<R>> &&
        indirect_binary_predicate<ranges::equal_to,
                                projected<iterator_t<R>, Proj>, const T*>
    borrowed_subrange_t<R>
        ranges::remove(Ep&& exec, R&& r, const T& value, Proj proj = {});
[...]
```

```
template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
    requires permutable<I>
    subrange<I>
        ranges::remove_if(Ep& exec, I first, S last, Pred pred, Proj proj = {});
template<execution-policy Ep, sized_random_access_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    requires permutable<iterator_t<R>>
    borrowed_subrange_t<R>
        ranges::remove_if(Ep& exec, R& r, Pred pred, Proj proj = {});
```

-1- Let E be [...]

3. Modify 26.8.5 [\[alg.partitions\]](#) as indicated:

```
[...]
template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
    requires permutable<I>
    subrange<I>
        ranges::partition(Ep&& exec, I first, S last, Pred pred, Proj proj = {});
template<execution-policy Ep, sized_random_access_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    requires permutable<iterator_t<R>>
    borrowed_subrange_t<R>
        ranges::partition(Ep&& exec, R&& r, Pred pred, Proj proj = {});
```

-1- Let $proj$ be `identity{}` for the overloads with no parameter named $proj$.

4299. Missing *Mandates*: part in `optional<T&>::transform`

Section: 22.5.4.7 [\[optional.ref.monadic\]](#) **Status:** [Voting](#) **Submitter:** Giuseppe D'Angelo **Opened:** 2025-07-15 **Last modified:** 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [\[optional.ref.monadic\]](#).

View all other [issues](#) in [\[optional.ref.monadic\]](#).

Discussion:

In 22.5.4.7 [\[optional.ref.monadic\]](#) the specification of `optional<T&>::transform` is missing an additional part of the *Mandates*: element compared to the primary template's transform (in 22.5.3.8 [\[optional.monadic\]](#) p8); that is, is missing to enforce that the U type is a valid contained type for `optional`.

The definition of "valid contained type" comes from [P2988R12](#). The paper amended the *Mandates*: element of the primary template's transform to use this definition. The fact that the same wording has not been applied to `optional<T&>::transform` as well looks like an oversight. I would suggest to apply it.

[2025-10-16; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

As `optional<remove_cv_t<invoke_result_t<F, T&>>>` is part of the signature (return type), we never enter the body to trigger the *Mandates*, so it's already implicitly ill-formed if the result of f is not a valid contained type. It's worth clarifying that though."

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 22.5.4.7 [\[optional.ref.monadic\]](#) as indicated:

```
template<class F>
    constexpr optional<remove_cv_t<invoke_result_t<F, T&>>> transform(F&& f) const;
```

-4- Let U be `remove_cv_t<invoke_result_t<F, T&>>`.

-5- *Mandates*: `U is a valid contained type for optional`. The declaration

```
U u(invoke(std::forward<F>(f), *val ));
```

is well-formed for some invented variable u.

4300. Missing *Returns*: element in optional<T&>::emplace

Section: 22.5.4.3 [\[optional.ref.assign\]](#) **Status:** [Voting](#) **Submitter:** Giuseppe D'Angelo **Opened:** 2025-07-15 **Last modified:** 2025-10-30

Priority: Not Prioritized

Discussion:

The specification for optional<T&>::emplace in 22.5.4.3 [\[optional.ref.assign\]](#) is not specifying the returned value via a *Returns*: element; however the function does indeed return something (a T&). Such a *Returns*: element is there for the primary template's emplace (cf. 22.5.3.4 [\[optional.assign\]](#)).

[2025-08-27; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 22.5.4.3 [\[optional.ref.assign\]](#) as indicated:

```
template<class U>
constexpr T& emplace(U&& u) noexcept(is_nothrow_constructible_v<T&, U>);

-4- Constraints: [...]

-5- Effects: Equivalent to: convert-ref-init-val(std::forward<U>(u)).

-?- Returns: *val.
```

4301. condition_variable_{_any}::wait_{for, until} should take timeout by value

Section: 32.7.4 [\[thread.condition.condvar\]](#), 32.7.5 [\[thread.condition.condvarany\]](#) **Status:** [Voting](#) **Submitter:** Hui Xie **Opened:** 2025-07-19 **Last modified:** 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [thread.condition.condvar].

View all other [issues](#) in [thread.condition.condvar].

Discussion:

At the moment, both condition_variable and condition_variable_any's wait_for and wait_until take the timeout time_point/duration by const reference. This can cause surprising behaviour. Given the following example (thanks to Tim Song):

```
struct Task {
    system_clock::time_point deadline;
    // stuff
};

std::mutex mtx;
std::condition_variable cv;
std::priority_queue<Task, vector<Task>, CompareDeadlines> queue;

// thread 1
std::unique_lock lck(mtx);
if (queue.empty()) { cv.wait(lck); }
else { cv.wait_until(lck, queue.top().deadline); }

// thread 2
std::lock_guard lck(mtx);
queue.push(/* some task */);
cv.notify_one();
```

From the user's point of view, it is sufficiently locked on both threads. However, due to the fact that the time_point is taken by reference, and that both libc++ and libstdc++'s implementation will read the value again after waking up, this will read a dangling reference of the time_point.

Another example related to this issue:

We (libc++) recently received a bug report on condition_variable_{_any}::wait_{for, until}.

Basically the user claims that these functions take time_point/duration by const reference, if the user modifies the time_point/duration on another thread with the same mutex, they can get unexpected return value for condition_variable, and data race for conditional_variable_any.

Bug report [here](#).

Reproducer (libstdc++ has the same behaviour as ours) [on godbolt](#).

```
std::mutex mutex;
std::condition_variable cv;
auto timeout = std::chrono::steady_clock::time_point::max();

// Thread 1:
std::unique_lock lock(mutex);
const auto status = cv.wait_until(lock, timeout);

// Thread 2:
std::unique_lock lock(mutex);
cv.notify_one();
timeout = std::chrono::steady_clock::time_point::min();
```

So basically the problem was that when we return whether there is no_timeout or timeout at the end of the function, we read the const reference again, which can be changed since the beginning of the function. For condition_variable, it is "unexpected results" according to the user. And in conditional_variable_any, we actually unlock the user lock and acquire our internal lock, then read the input again, so this is actually a data race.

For wait_for, the spec has

Effects: Equivalent to: return wait_until(lock, chrono::steady_clock::now() + rel_time);

So the user can claim our implementation is not conforming because the spec says there needs to be a temporary time_point (now + duration) created and since it should operate on this temporary time_point. There shouldn't be any unexpected behaviour or data race.

For wait_until it is unclear whether the spec has implications that implementations are allowed to read abs_time while the user's lock is unlocked.

it is also unclear if an implementation is allowed to return timeout if cv indeed does not wait longer than the original value of timeout. If it is not allowed, implementations will have to make a local copy of the input rel_time or abs_time, which defeats the purpose of taking arguments by const reference.

For both of the examples, Ville has a great comment in the reflector:

It seems like a whole bag of problems goes away if these functions just take the timeout by value?

libc++ implementers have strong preference just changing the API to take these arguments by value, and it is not an ABI break for us as the function signature has changed.

[2025-08-29; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 32.7.4 [\[thread.condition.condvar\]](#) as indicated:

```
namespace std {
    class condition_variable {
    public:
        [...]
        template<class Predicate>
            void wait(unique_lock<mutex>& lock, Predicate pred);
        template<class Clock, class Duration>
            cv_status wait_until(unique_lock<mutex>& lock,
                               const chrono::time_point<Clock, Duration>& abs_time);
        template<class Clock, class Duration, class Predicate>
            bool wait_until(unique_lock<mutex>& lock,
                           const chrono::time_point<Clock, Duration>& abs_time,
                           Predicate pred);
        template<class Rep, class Period>
            cv_status wait_for(unique_lock<mutex>& lock,
                              const chrono::duration<Rep, Period>& rel_time);
        template<class Rep, class Period, class Predicate>
            bool wait_for(unique_lock<mutex>& lock,
                          const chrono::duration<Rep, Period>& rel_time,
                          Predicate pred);
        [...]
    };
}

[...]
```

```
template<class Clock, class Duration>
    cv_status wait_until(unique_lock<mutex>& lock,
                        const chrono::time_point<Clock, Duration>& abs_time);

-17- Preconditions: [...]
```

```
[...]
```

```
template<class Rep, class Period>
    cv_status wait_for(unique_lock<mutex>& lock,
                      const chrono::duration<Rep, Period>& rel_time);
```

-23- *Preconditions*: [...]

[...]

```
template<class Clock, class Duration, class Predicate>
bool wait_until(unique_lock<mutex>& lock,
               const chrono::time_point<Clock, Duration>& abs_time,
               Predicate pred);
```

-29- *Preconditions*: [...]

[...]

```
template<class Rep, class Period, class Predicate>
bool wait_for(unique_lock<mutex>& lock,
             const chrono::duration<Rep, Period>& rel_time,
             Predicate pred);
```

-35- *Preconditions*: [...]

[...]

2. Modify 32.7.5.1 [\[thread.condition.condvarany.general\]](#) as indicated:

```
namespace std {
class condition_variable_any {
public:
    [...]
    // 32.7.5.2 \[thread.condvarany.wait\], noninterruptible waits
    template<class Lock>
    void wait(Lock& lock);
    template<class Lock, class Predicate>
    void wait(Lock& lock, Predicate pred);

    template<class Lock, class Clock, class Duration>
    cv_status wait_until(Lock& lock, const chrono::time_point<Clock, Duration>& abs_time);
    template<class Lock, class Clock, class Duration, class Predicate>
    bool wait_until(Lock& lock, const chrono::time_point<Clock, Duration>& abs_time,
                   Predicate pred);
    template<class Lock, class Rep, class Period>
    cv_status wait_for(Lock& lock, const chrono::duration<Rep, Period>& rel_time);
    template<class Lock, class Rep, class Period, class Predicate>
    bool wait_for(Lock& lock, const chrono::duration<Rep, Period>& rel_time, Predicate pred);

    // 32.7.5.3 \[thread.condvarany.intwait\], interruptible waits
    template<class Lock, class Predicate>
    bool wait(Lock& lock, stop_token stoken, Predicate pred);
    template<class Lock, class Clock, class Duration, class Predicate>
    bool wait_until(Lock& lock, stop_token stoken,
                   const chrono::time_point<Clock, Duration>& abs_time, Predicate pred);
    template<class Lock, class Rep, class Period, class Predicate>
    bool wait_for(Lock& lock, stop_token stoken,
                 const chrono::duration<Rep, Period>& rel_time, Predicate pred);
};
}
```

3. Modify 32.7.5.2 [\[thread.condvarany.wait\]](#) as indicated:

[...]

```
template<class Lock, class Clock, class Duration>
cv_status wait_until(Lock& lock, const chrono::time_point<Clock, Duration>& abs_time);
```

-6- *Effects*: [...]

[...]

```
template<class Lock, class Rep, class Period>
cv_status wait_for(Lock& lock, const chrono::duration<Rep, Period>& rel_time);
```

-11- *Effects*: [...]

[...]

```
template<class Lock, class Clock, class Duration, class Predicate>
bool wait_until(Lock& lock, const chrono::time_point<Clock, Duration>& abs_time,
               Predicate pred);
```

-16- *Effects*: [...]

[...]

```
template<class Lock, class Rep, class Period, class Predicate>
bool wait_for(Lock& lock, const chrono::duration<Rep, Period>& rel_time, Predicate pred);
```

-19- *Effects*: [...]

4. Modify 32.7.5.3 [\[thread.condvarany.intwait\]](#) as indicated:

```
[...]
template<class Lock, class Clock, class Duration, class Predicate>
bool wait_until(Lock& lock, stop_token stoken,
               const chrono::time_point<Clock, Duration>s abs_time, Predicate pred);

-7- Effects: [...]

[...]

template<class Lock, class Rep, class Period, class Predicate>
bool wait_for(Lock& lock, stop_token stoken,
              const chrono::duration<Rep, Period>s rel_time, Predicate pred);

-13- Effects: [...]
```

4305. Missing user requirements on type_order template

Section: 17.12.7 [\[compare.type\]](#) **Status:** [Voting](#) **Submitter:** Daniel Krüger **Opened:** 2025-07-27 **Last modified:** 2025-10-30

Priority: Not Prioritized

Discussion:

The recently approved paper [P2830R10](#) proposes to add the new type_order type traits to 17.12 [\[cmp\]](#) (and thus outside of 21.3 [\[type.traits\]](#)), which has the subtle and most likely unintended effect, that it doesn't fall under the general requirement expressed in 21.3.2 [\[meta.rqmts\]](#) p4,

Unless otherwise specified, the behavior of a program that adds specializations for any of the templates specified in 21.3.2 [\[meta.rqmts\]](#) is undefined.

and so in principle the explicit allowance specified in 16.4.5.2.1 [\[namespace.std\]](#) p2,

Unless explicitly prohibited, a program may add a template specialization for any standard library class template to namespace std [...]

holds. So we need to add extra wording to the type_order specification in 17.12.7 [\[compare.type\]](#) to prohibit such program specializations.

This was reported shortly before the Sofia meeting [during reflector discussion](#) but seems to be forgotten before the final paper appeared on plenary.

During the reflector discussion two possible ways to solve this issue were pointed out:

1. The most simple one would mimic the wording in 21.3.2 [\[meta.rqmts\]](#) p4 quoted above.
2. Instead of introducing just another undefined opportunity to run into undefined behaviour it has been pointed out that we could follow the approach taken by std::initializer_list and make the program ill-formed in this case, as specified in 17.11.2 [\[initializer.list.syn\]](#) p2:

If an explicit specialization or partial specialization of initializer_list is declared, the program is ill-formed.

Jonathan Wakely responded to the reflector discussion:

I think ill-formed would be better. It shouldn't be difficult for implementations to have special cases that are disallowed.

Given the already existing experience with std::initializer_list the proposed wording below therefore follows the ill-formed program approach.

[2025-10-14; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 17.12.7 [\[compare.type\]](#) as indicated:

```
template<class T, class U>
struct type_order;

-2- The name type_order denotes a Cpp17BinaryTypeTrait (21.3.2 \[meta.rqmts\]) with a base characteristic of
integral_constant<strong_ordering, TYPE-ORDER(T, U)>.

-?- If an explicit specialization or partial specialization of type_order is declared, the program is ill-formed.

-3- Recommended practice: The order should be lexicographical on parameter-type-lists and template argument lists.
```

4312. Const and value category mismatch for allocator_arg_t/allocator_arg in the description of uses-allocator construction

Section: 20.2.8.2 [\[allocator.uses.construction\]](#) **Status:** [Voting](#) **Submitter:** Jiang An **Opened:** 2025-08-06 **Last modified:** 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [allocator.uses.construction].

View all other [issues](#) in [allocator.uses.construction].

Discussion:

Currently, 20.2.8.2 [\[allocator.uses.construction\]](#) bullet 2.2 states:

Otherwise, if T has a constructor invocable as T(allocator_arg, alloc, args...) (leading-allocator convention), [...]

However, when forming construction arguments in the utility functions, we're actually using cv-unqualified rvalue of allocator_arg_t, which can be inferred from using plain allocator_arg_t but not const allocator_arg_t& in 20.2.8.2 [\[allocator.uses.construction\]](#) bullet 5.2.

It seems that such mismatch was present even since C++11 (per [N3337](#) [allocator.uses.construction]/1.2). If the use of plain allocator_arg_t is considered correct, I think we should fix the description.

[2025-10-14; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Unless the std::allocator_arg tag object is not supposed to be used, wouldn't it make more sense to preserve the "if T has a constructor invocable as T(allocator_arg, alloc, args...)" wording and change every allocator_arg_t into const allocator_arg_t&, so that we check for construction from the const tag object, and then actually use a const value in the constructor arguments. Strongly don't care though.

Proposed resolution:

This wording is relative to [N5014](#).

- 1. Modify 20.2.8.2 [\[allocator.uses.construction\]](#) as indicated:
 - 2- The following utility functions support three conventions for passing alloc to a constructor:
 - (2.1) — [...]
 - (2.2) — Otherwise, if T has a constructor invocable as T(~~allocator_arg~~[allocator_arg_t{}](#), alloc, args...) (leading-allocator convention), then uses-allocator construction chooses this constructor form.
 - (2.3) — [...]

4317. The meaning of "resource" in the Cpp17Destructible requirements is undefined

Section: 16.4.4.2 [\[utility.arg.requirements\]](#) **Status:** [Voting](#) **Submitter:** Jiang An **Opened:** 2025-08-15 **Last modified:** 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [utility.arg.requirements].

View all other [issues](#) in [utility.arg.requirements].

Discussion:

The meaning of "resource" in the Cpp17Destructible requirements cannot be inferred from the standard wording and it seems unlikely that the standard will determine its meaning in the future. What are considered as resources generally depends on users' intent, so the standard shouldn't determine the well-definedness of a program execution due to it. Moreover, the wording doesn't seem to consider shared ownership, which can be represented by shared_ptr.

[2025-10-14; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

- 1. Modify 16.4.4.2 [\[utility.arg.requirements\]](#), Table 35 [tab:cpp17.destructible] as indicated:

Table 35 — Cpp17Destructible requirements [tab:cpp17.destructible]	
Expression	Post-condition
u.~T()	All resources owned by u are reclaimed. No exception is propagated.
[Note 3: Array types and non-object types are not Cpp17Destructible. — end note]	

4318. Have hive::erase_if reevaluate end() to avoid UB

Section: 23.3.9.6 [\[hive.erasure\]](#) **Status:** [Voting](#) **Submitter:** Frank Birbacher **Opened:** 2025-08-16 **Last modified:** 2025-10-30

Priority: Not Prioritized

View all other [issues](#) in [hive.erase].

Discussion:

Background: <https://github.com/cplusplus/draft/pull/8162>

For 23.3.9.6 [\[hive.erase\]](#) p2, the defining code must not cache the end-iterator. In case the last element of the sequence is removed, the past-the-end iterator will be invalidated. This will trigger UB in the loop condition. Instead, re-evaluate `end()` each time.

[2025-08-29; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

[Drafting note: There are other ways to fix this code while keeping the caching behaviour, but I don't see any particular reason to do so for the definition of the effects.]

1. Modify 23.3.9.6 [\[hive.erase\]](#) as indicated:

```
template<class T, class Allocator, class Predicate>
typename hive<T, Allocator>::size_type
erase_if(hive<T, Allocator>& c, Predicate pred);
```

-2- *Effects*: Equivalent to:

```
auto original_size = c.size();
for (auto i = c.begin(); last = c.end();; i != lastc.end\(\); ) {
    if (pred(*i)) {
        i = c.erase(i);
    } else {
        ++i;
    }
}
return original_size - c.size();
```

[4328](#). Remove note in §[exec.sched] regarding waiting for completion of scheduled operations

Section: 33.6 [\[exec.sched\]](#) **Status:** [Voting](#) **Submitter:** Lewis Baker **Opened:** 2025-08-25 **Last modified:** 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [exec.sched].

View all other [issues](#) in [exec.sched].

Discussion:

The note at the end of 33.6 [\[exec.sched\]](#) says:

[Note: The ability to wait for completion of submitted function objects can be provided by the associated execution resource of the scheduler —
end note]

The suggestion that the execution resource should be used to join/wait on scheduled work is problematic in situations that may involve more than one execution context, as an execution resource having an empty queue of scheduled work does not necessarily imply that tasks currently running on another execution context will not later schedule additional work on this execution resource.

With the introduction of `counting_scope` with [P3149](#) we now have a better recommended way of waiting for tasks that use a resource (including execution resources) to complete.

The note as it stands represents bad guidance and should either be removed or updated to refer to `counting_scope` and `simple_counting_scope` (33.14.2 [\[exec.counting.scopes\]](#)).

[2025-10-23; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

"What is this telling me as a user? That a custom execution resource can add a non-standard 'wait for completion' facility?"

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 33.6 [\[exec.sched\]](#) as indicated:

-7- A scheduler type's destructor shall not block pending completion of any receivers connected to the sender objects returned from schedule.

~~[Note 1: The ability to wait for completion of submitted function objects can be provided by the associated execution resource of the scheduler. — end note]~~

4340. task::promise_type::unhandled_stopped() should be noexcept

Section: 33.13.6.5 [\[task.promise\]](#) **Status:** [Voting](#) **Submitter:** Dietmar Kühl **Opened:** 2025-08-31 **Last modified:** 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [task.promise].

View all other [issues](#) in [task.promise].

Discussion:

Addresses [US 252-387](#)

The function `task::promise_type::unhandled_stopped()` is called from `set_stopped()` of a receiver and calls `set_stopped` itself. These functions are required to be `noexcept`. Thus, `unhandled_stopped()` can't throw an exception and should be marked `noexcept`. All other declarations of `unhandled_stopped()` are already marked `noexcept` but `task::promise_type::unhandled_stopped()` isn't.

[2025-10-17; Reflector poll.]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

In the synopsis in 33.13.6.5 [\[task.promise\]](#) add `noexcept` to the declaration of `task::promise_type::unhandled_stopped()`:

```
namespace std::execution {
  template<class T, class Environment>
  class task<T, Environment>::promise_type {
    ...
    coroutine_handle<> unhandled_stopped() noexcept;
    ...
  };
}
```

In the specification in 33.13.6.5 [\[task.promise\]](#) paragraph 13 add `noexcept`:

```
coroutine_handle<> unhandled_stopped() noexcept;
```

-13- *Effects*: Completes the asynchronous operation associated with `STATE(*this)` by invoking `set_stopped(std::move(RCVR(*this)))`.

4341. Missing rvalue reference qualification for task::connect()

Section: 33.13.6.2 [\[task.class\]](#) **Status:** [Voting](#) **Submitter:** Dietmar Kühl **Opened:** 2025-08-31 **Last modified:** 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [task.class].

View all other [issues](#) in [task.class].

Discussion:

Addresses [US 244-375](#)

Coroutines can't be copied. Thus, a task can be `connect()` just once. To represent that `task::connect()` should be rvalue reference qualified but currently it isn't.

[2025-10-17; Reflector poll.]

Set status to Tentatively Ready after five votes in favour during reflector poll.

"It's nice to rvalue qualify such a function, but it is not strictly necessary."

Proposed resolution:

In the synopsis in 33.13.6.2 [\[task.class\]](#) add rvalue reference qualification to `task::connect()`:

```
namespace std::execution {
  template<class T, class Environment>
  class task {
    ...
    template<receiver Rcvr>
    state<Rcvr> connect(Rcvr&& rcvr) &&;
```

```
    } ...
}
```

In the specification in 33.13.6.3 [\[task.members\]](#) paragraph 3 add rvalue reference qualification to `task::connect()`:

```
template<receiver Rcvr>
    state<Rcvr> connect(Rcvr&& rcvr) &&;
```

-3- *Precondition*: `bool(handle)` is true.

-4- *Effects*: Equivalent to:

```
return state<Rcvr>(exchange(handle, {}), std::forward<Rcvr>(rcvr));
```

4342. Missing rvalue reference qualification for `task_scheduler::ts-sender::connect()`

Section: 33.13.5 [\[exec.task.scheduler\]](#) **Status:** [Voting](#) **Submitter:** Dietmar Kühl **Opened:** 2025-09-01 **Last modified:** 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in `[exec.task.scheduler]`.

View all other [issues](#) in `[exec.task.scheduler]`.

Discussion:

Addresses [US 237-369](#)

The result of `schedule(sched)` for a scheduler `sched` is only required to be movable. An object of this type may need to be forwarded to an operation state constructor by `task_scheduler::ts-sender::connect`. Thus, this function should be qualified with an rvalue reference.

[2025-10-17; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

Add an rvalue qualifier to the declaration of `connect` in 33.13.5 [\[exec.task.scheduler\]](#) paragraph 8:

```
namespace std::execution {
    class task_scheduler::ts-sender {    // exposition only
    public:
        using sender_concept = sender_t;

        template<receiver Rcvr>
            state<Rcvr> connect(Rcvr&& rcvr) &&;
    };
}
```

In the specification in 33.13.5 [\[exec.task.scheduler\]](#) paragraph 10 add an rvalue qualifier to `connect`:

```
template<receiver Rcvr>
    state<Rcvr> connect(Rcvr&& rcvr) &&;
```

-10- *Effects*: Let r be an object of a type that models receiver and whose completion handlers result in invoking the corresponding completion handlers of `rcvr` or copy thereof. Returns an object of type `state<Rcvr>` containing an operation state object initialized with `connect(SENDER(*this), std::move(r))`.

4343. Missing default template arguments for `task`

Section: 33.13.6.2 [\[task.class\]](#) **Status:** [Voting](#) **Submitter:** Dietmar Kühl **Opened:** 2025-09-01 **Last modified:** 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in `[task.class]`.

View all other [issues](#) in `[task.class]`.

Discussion:

Addresses [US 243-376](#)

The design discussion of `task` describes defaults for the two template parameters `T` and `Environment` of `task` but these defaults are not reflected in the synopsis of 33.13.6.2 [\[task.class\]](#). This is an oversight and should be fixed. The default for `T` should be `void` and the default for `Environment` should be `env<>` (the design paper used `empty_env` but this struct was replaced by the class template `env` by [P3325R5](#)).

There could be a counter argument to defining a default for the `Environment` template parameter: this type is used to determine various customizations of `task`, e.g., the `allocator_type`, the `scheduler_type`, and the `stop_source_type`. Leaving the type a required argument means that a future standard could choose a

possibly better default than the types determined when the Environment doesn't define them. On the other hand, a future standard could provide a suitable alias with modified types under a different name and/or a different namespace. Based on the LEWG discussion on 2025-08-26 the direction is to add the default arguments.

[2025-10-17; Reflector poll.]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

Add default template arguments for task for T = void and Environment = env<> in the synopsis of 33.13.6.2 [\[task.class\]](#):

```
namespace std::execution {
    template<class T = void, class Environment = env<>>
    class task {
    ...
    };
}
```

4345. task::promise_type::return_value default template parameter

Section: 33.13.6.5 [\[task.promise\]](#) Status: [Voting](#) Submitter: Dietmar Kühl Opened: 2025-09-01 Last modified: 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [task.promise].

View all other [issues](#) in [task.promise].

Discussion:

Addresses [US 251-388](#)

The template parameter V of task::promise_type::return_value doesn't have a default template argument specified. Specifying a default template argument of T would enable use of co_return { ... } which would be consistent with normal return statements. This feature was not discussed in the design paper but based on the LEWG discussion on 2025-08-26 it is considered to be more a bug fix than a new feature.

[2025-10-17; Reflector poll.]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

Add a default template argument of T to the template parameter V of task::promise_type::return_value in the synopsis of 33.13.6.5 [\[task.promise\]](#):

```
namespace std::execution {
    template<class T, class Environment>
    class task<T, Environment>::promise_type {
    ...
        template<typename V = T>
        void return_value(V&& value);
    ...
    };
}
```

4346. task::promise_type::return_void/value lack a specification

Section: 33.13.6.5 [\[task.promise\]](#) Status: [Voting](#) Submitter: Dietmar Kühl Opened: 2025-09-01 Last modified: 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [task.promise].

View all other [issues](#) in [task.promise].

Discussion:

Addresses [US 250-389](#)

The synopsis for std::execution::task<T, E>::promise_type declares return_void() or return_value(V&&). However, there is no specification of what these functions actually do. return_void() doesn't need to do anything at all. return_value(V&& v) needs to store v into the result.

[2025-10-23; Reflector poll.]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

Insert the following paragraphs in 33.13.6.5 [\[task.promise\]](#) after the specification of `unhandled_stopped`:

```
coroutine_handle<> unhandled_stopped();
```

-13- *Effects*: Completes the asynchronous operation associated with `STATE(*this)` by invoking `set_stopped(std::move(RCVR(*this)))`.

-14- *Returns*: `noop_coroutine()`.

```
void return_void();
```

?- Effects: does nothing.

```
template<class V>
void return_value(V&& v);
```

?- Effects: Equivalent to `result.emplace(std::forward<V>(v))`.

```
unspecified get_env() const noexcept;
```

-15- *Returns*: An object `env` such that queries are forwarded as follows:

4349. task is not actually started lazily

Section: 33.13.6.5 [\[task.promise\]](#) **Status:** [Voting](#) **Submitter:** Dietmar Kühl **Opened:** 2025-09-01 **Last modified:** 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [\[task.promise\]](#).

View all other [issues](#) in [\[task.promise\]](#).

Discussion:

Addresses [US 258-381](#)

The wording for `task<...>::promise_type::initial_suspend` in 33.13.6.5 [\[task.promise\]](#) paragraph 6 (second bullet) may imply that a task is eagerly started, i.e., that the awaiter return from `initial_suspend()` immediately starts the scheduling operation and cause the task to be resumed. At the very least the second bullet of the wording should be clarified such that the scheduling operation is only started when the coroutine gets resumed.

An alternative resolution it have `initial_suspend()` return `std::suspend_always` implicitly requiring that the task gets `start()`ed from the correct execution context. This approach has the advantage of avoiding unnecessary scheduling operations for the likely common case when tasks are started from the correct context.

[2025-10-18; Reflector poll.]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

Change the declaration of `initial_suspend()` in the synopsis of 33.13.6.5 [\[task.promise\]](#) to use `suspend_always`, directly provide a definition, and add various qualifiers:

```
namespace std::execution {
    template<class T, class Environment>
    class task<T, Environment>::promise_type {
        ...
        auto static constexpr suspend_always initial_suspend() noexcept { return {}; }
        ...
    };
}
```

Remove 33.13.6.5 [\[task.promise\]](#) paragraph 6 entirely:

~~auto initial_suspend() noexcept;~~

~~-6- *Returns*: An awaitable object of unspecified type (`{expr.await}`) whose member functions arrange for~~

~~-6.1- the calling coroutine to be suspended;~~

~~-6.2- the coroutine to be resumed on an execution agent of the execution resource associated with `SCHED(*this)`.~~

4351. integral-constant-like needs more `remove_cvref_t`

Section: 23.7.2.1 [\[span.syn\]](#) **Status:** [Voting](#) **Submitter:** Jonathan Wakely **Opened:** 2025-09-05 **Last modified:** 2025-10-30

Priority: Not Prioritized

Discussion:

[P2781R9](#) tweaked the definition of *integral-constant-like* to work with `constant_wrapper`, like so:

```
template<class T>
concept integral-constant-like = // exposition only
    is_integral_v<remove_cvref_t<decltype(T::value)>> &&
    !is_same_v<bool, remove_const_t<decltype(T::value)>> &&
    convertible_to<T, decltype(T::value)> &&
    equality_comparable_with<T, decltype(T::value)> &&
    bool_constant<T() == T::value::value &&
    bool_constant<static_cast<decltype(T::value)>(T()) == T::value::value;
```

This was done so that `cw<5>` models the concept, but it needs an additional tweak so that `cw<true>` does not model it.

[2025-10-17; Reflector poll.]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 23.7.2.1 [\[span.syn\]](#) as indicated:

```
template<class T>
concept integral-constant-like = // exposition only
    is_integral_v<remove_cvref_t<decltype(T::value)>> &&
    !is_same_v<bool, remove_cvref_const_t<decltype(T::value)>> &&
    convertible_to<T, decltype(T::value)> &&
    equality_comparable_with<T, decltype(T::value)> &&
    bool_constant<T() == T::value::value &&
    bool_constant<static_cast<decltype(T::value)>(T()) == T::value::value;
```

[4366](#). Heterogeneous comparison of expected may be ill-formed

Section: 22.8.6.8 [\[expected.object.eq\]](#), 22.8.7.8 [\[expected.void.eq\]](#) **Status:** [Voting](#) **Submitter:** Hewill Kang **Opened:** 2025-09-06 **Last modified:** 2025-10-30

Priority: Not Prioritized

Discussion:

These comparison functions all explicitly `static_cast` the result of the underlying comparison to `bool`. However, the *Constraints* only require the implicit conversion, not the explicit one (i.e., "convertible to `bool`" rather than "models *boolean-testable*").

This means that in some pathological cases it will lead to hard errors ([demo](#)):

```
#include <expected>

struct E1 {};
struct E2 {};

struct Bool {
    operator bool() const;
    explicit operator bool() = delete;
};
Bool operator==(E1, E2);

int main() {
    std::unexpected e1{E1{}};
    std::unexpected e2{E2{}};
    return std::expected<int, E1>{e1} == e2; // fire
}
```

It is reasonable to specify return consistency with actual *Constraints*.

[2025-10-16; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Related to LWG [4366^{\(i\)}](#), but the wording styles are inconsistent. `optional` uses "Effects: Equivalent to ..." and `expected` just uses *Returns*..

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 22.8.6.8 [\[expected.object.eq\]](#) as indicated:

```
template<class T2> friend constexpr bool operator==(const expected& x, const T2& v);
```

-3- *Constraints*: T2 is not a specialization of `expected`. The expression `*x == v` is well-formed and its result is convertible to `bool`.

[Note 1: T need not be *Cpp17EqualityComparable*. — end note]

-4- Returns: `if x.has_value() is true, && static_cast<bool>(*x == v); otherwise false.`

```
template<class E2> friend constexpr bool operator==(const expected& x, const unexpected<E2>& e);
```

-5- Constraints: The expression `x.error() == e.error()` is well-formed and its result is convertible to `bool`.

-6- Returns: `if !x.has_value() is true, && static_cast<bool>(x.error() == e.error()); otherwise false.`

2. Modify 22.8.7.8 [\[expected_void_eq\]](#) as indicated:

```
template<class T2, class E2> requires is_void_v<T2>
friend constexpr bool operator==(const expected& x, const expected<T2, E2>& y);
```

-1- Constraints: The expression `x.error() == y.error()` is well-formed and its result is convertible to `bool`.

-2- Returns: If `x.has_value()` does not equal `y.has_value()`, `false`; otherwise `if x.has_value() is true, true; otherwise static_cast<bool>(x.error() == y.error());` `++`

```
template<class E2>
friend constexpr bool operator==(const expected& x, const unexpected<E2>& e);
```

-3- Constraints: The expression `x.error() == e.error()` is well-formed and its result is convertible to `bool`.

-4- Returns: `if !x.has_value() is true, && static_cast<bool>(x.error() == e.error()); otherwise false.`

4370. Comparison of `optional<T>` to `T` may be ill-formed

Section: 22.5.9 [\[optional_comp_with_t\]](#) Status: [Voting](#) Submitter: Hewill Kang Opened: 2025-09-06 Last modified: 2025-10-30

Priority: Not Prioritized

View all other [issues](#) in [\[optional.comp.with.t\]](#).

Discussion:

When comparing an `optional` with its value type, the current wording specifies that the result is the ternary expression of `x.has_value() ? *x == v : false`, where `*x == v` returns a result that can be implicitly converted to `bool`.

However, when the result can also be constructed using `bool` (which is common), the ternary operation will be ill-formed due to ambiguity ([demo](#)):

```
#include <optional>

struct Bool {
    Bool(bool);
    operator bool() const;
};

struct S {
    Bool operator==(S) const;
};

int main() {
    return std::optional<S>{} == S{}; // fire
}
```

[2025-10-16; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

"Alternatively could keep the conditional operator but cast one side to `bool`, but that would do an explicit conversion, which might not be what we want."

"Should just require *boolean-testable*."

Related to LWG [4366](#)⁽¹⁾.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 22.5.9 [\[optional_comp_with_t\]](#) as indicated:

```
template<class T, class U> constexpr bool operator==(const optional<T>& x, const U& v);
```

-1- Constraints: U is not a specialization of `optional`. The expression `*x == v` is well-formed and its result is convertible to `bool`.

[Note 1: T need not be *Cpp17EqualityComparable*. — end note]

-2- Effects: Equivalent to: `return x.has_value() ? *x == v : false;`


```

if (x.has_value())
    return *x == v;
return false;

```

template<class T, class U> constexpr bool operator==(const T& v, const optional<U>& x);

-3- *Constraints*: T is not a specialization of optional. The expression `v == *x` is well-formed and its result is convertible to bool.

-4- *Effects*: Equivalent to: ~~return x.has_value() ? v == *x : false;~~

```

if (x.has_value())
    return v == *x;
return false;

```

template<class T, class U> constexpr bool operator!=(const optional<T>& x, const U& v);

-5- *Constraints*: U is not a specialization of optional. The expression `*x != v` is well-formed and its result is convertible to bool.

-6- *Effects*: Equivalent to: ~~return x.has_value() ? *x != v : true;~~

```

if (x.has_value())
    return *x != v;
return true;

```

template<class T, class U> constexpr bool operator!=(const T& v, const optional<U>& x);

-7- *Constraints*: T is not a specialization of optional. The expression `v != *x` is well-formed and its result is convertible to bool.

-8- *Effects*: Equivalent to: ~~return x.has_value() ? v != *x : true;~~

```

if (x.has_value())
    return v != *x;
return true;

```

template<class T, class U> constexpr bool operator<(const optional<T>& x, const U& v);

-9- *Constraints*: U is not a specialization of optional. The expression `*x < v` is well-formed and its result is convertible to bool.

-10- *Effects*: Equivalent to: ~~return x.has_value() ? *x < v : true;~~

```

if (x.has_value())
    return *x < v;
return true;

```

template<class T, class U> constexpr bool operator<(const T& v, const optional<U>& x);

-11- *Constraints*: T is not a specialization of optional. The expression `v < *x` is well-formed and its result is convertible to bool.

-12- *Effects*: Equivalent to: ~~return x.has_value() ? v < *x : false;~~

```

if (x.has_value())
    return v < *x;
return false;

```

template<class T, class U> constexpr bool operator>(const optional<T>& x, const U& v);

-13- *Constraints*: U is not a specialization of optional. The expression `*x > v` is well-formed and its result is convertible to bool.

-14- *Effects*: Equivalent to: ~~return x.has_value() ? *x > v : false;~~

```

if (x.has_value())
    return *x > v;
return false;

```

template<class T, class U> constexpr bool operator>(const T& v, const optional<U>& x);

-15- *Constraints*: T is not a specialization of optional. The expression `v > *x` is well-formed and its result is convertible to bool.

-16- *Effects*: Equivalent to: ~~return x.has_value() ? v > *x : true;~~

```

if (x.has_value())
    return v > *x;
return true;

```

template<class T, class U> constexpr bool operator<=(const optional<T>& x, const U& v);

-17- *Constraints*: U is not a specialization of optional. The expression `*x <= v` is well-formed and its result is convertible to bool.

-18- *Effects*: Equivalent to: ~~return x.has_value() ? *x <= v : true;~~

```
if (x.has_value())
    return *x <= v;
return true;
```

```
template<class T, class U> constexpr bool operator<=(const T& v, const optional<U>& x);
```

-19- *Constraints*: T is not a specialization of optional. The expression `v <= *x` is well-formed and its result is convertible to bool.

-20- *Effects*: Equivalent to: ~~return x.has_value() ? v <= *x : false;~~

```
if (x.has_value())
    return v <= *x;
return false;
```

```
template<class T, class U> constexpr bool operator>=(const optional<T>& x, const U& v);
```

-21- *Constraints*: U is not a specialization of optional. The expression `*x >= v` is well-formed and its result is convertible to bool.

-22- *Effects*: Equivalent to: ~~return x.has_value() ? *x >= v : false;~~

```
if (x.has_value())
    return *x >= v;
return false;
```

```
template<class T, class U> constexpr bool operator>=(const T& v, const optional<U>& x);
```

-23- *Constraints*: T is not a specialization of optional. The expression `v >= *x` is well-formed and its result is convertible to bool.

-24- *Effects*: Equivalent to: ~~return x.has_value() ? v >= *x : true;~~

```
if (x.has_value())
    return v >= *x;
return true;
```

4372. Weaken *Mandates*: for dynamic padding values in padded layouts

Section: 23.7.3.4.8.1 [\[mdspan.layout.leftpad.overview\]](#), 23.7.3.4.9.1 [\[mdspan.layout.rightpad.overview\]](#) **Status:** [Voting](#) **Submitter:** Luc Grosheintz **Opened:** 2025-09-09 **Last modified:** 2025-10-30

Priority: Not Prioritized

Discussion:

Two new layouts were added to `<mdspan>` in C++26. Both have a template parameter `size_t PaddingValue`. This value is allowed to be `std::dynamic_extent` to signal that the padding value isn't known at compile time.

A class *Mandates*: element (in 23.7.3.4.8.1 [\[mdspan.layout.leftpad.overview\]](#) (5.2) and 23.7.3.4.9.1 [\[mdspan.layout.rightpad.overview\]](#) (5.2), respectively) requires (unconditionally) that

- `PaddingValue` is representable as a value of `index_type`.

Since `std::dynamic_extent` is defined as `size_t(-1)` (in 23.7.2.1 [\[span.syn\]](#)) this immediately prohibits all dynamically padded layout mappings for any `index_type` for which:

```
numeric_limit<index_type>::max() < numeric_limit<size_t>::max()
```

One example is `int` on a 64-bit system.

The proposed resolution states that the modified representability *Mandates*: element holds for `rank <= 1`, even though in that case the `PaddingValue` has no other effect. Hence, the *Mandates*: element could be weakened further.

[2025-10-17; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

"This matches the wording in 23.7.3.3.1 [\[mdspan.extents.overview\]](#) 1.2"

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 23.7.3.4.8.1 [\[mdspan.layout.leftpad.overview\]](#) as indicated:

-5- *Mandates*:

(5.1) — [...]

(5.2) — [if padding_value is not equal to dynamic_extent, then](#) padding_value is representable as a value of type index_type.

(5.3) — [...]

(5.4) — [...]

2. Modify 23.7.3.4.9.1 [\[mdspan.layout.rightpad.overview\]](#) as indicated:

-5- Mandates:

(5.1) — [...]

(5.2) — [if padding_value is not equal to dynamic_extent, then](#) padding_value is representable as a value of type index_type.

(5.3) — [...]

(5.4) — [...]

4375. std::simd::bit_ceil should not be noexcept

Section: 29.10.8.15 [\[simd.bit\]](#) **Status:** [Voting](#) **Submitter:** Matthias Kretz **Opened:** 2025-08-29 **Last modified:** 2025-10-30

Priority: Not Prioritized

Discussion:

std::simd::bit_ceil is declared 'noexcept' in 29.10.3 [\[simd.syn\]](#) and 29.10.8.15 [\[simd.bit\]](#). But

1. std::bit_ceil is not 'noexcept' (22.11.2 [\[bit.syn\]](#) and 22.11.5 [\[bit.pow.two\]](#)) and
2. std::simd::bit_ceil has a precondition.

[2025-10-22; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 29.10.3 [\[simd.syn\]](#), header <simd> synopsis, as indicated:

```
namespace std {  
    [...]  
    // 29.10.8.15 \[simd.bit\], bit manipulation  
    template<simd-vec-type V> constexpr V byteswap(const V& v) noexcept;  
    template<simd-vec-type V> constexpr V bit_ceil(const V& v) noexcept;  
    template<simd-vec-type V> constexpr V bit_floor(const V& v) noexcept;  
    [...]  
}
```

2. Modify 29.10.8.15 [\[simd.bit\]](#) as indicated:

```
template<simd-vec-type V> constexpr V bit_ceil(const V& v) noexcept;
```

-3- Constraints: The type V::value_type is an unsigned integer type (6.9.2 [\[basic.fundamental\]](#)).

-4- Preconditions: [...]

[...]

4377. Misleading note about lock-free property of std::atomic_ref

Section: 32.5.7.2 [\[atomics.ref.ops\]](#) **Status:** [Voting](#) **Submitter:** Brian Bi **Opened:** 2025-09-15 **Last modified:** 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [atomics.ref.ops].

View all other [issues](#) in [atomics.ref.ops].

Discussion:

Note 1 to 32.5.7.2 [\[atomics.ref.ops\]](#) states:

Hardware could require an object referenced by an atomic_ref to have stricter alignment (6.8.3 [\[basic.align\]](#)) than other objects of type T. Further, whether operations on an atomic_ref are lock-free could depend on the alignment of the referenced object. For example, lock-free operations on std::complex<double> could be supported only if aligned to 2*alignof(double).

By using the word "Further", the note misleadingly implies that `required_alignment` may need to be greater than `alignof(T)` even before considering lock freedom, i.e., that `std::atomic_ref<T>` may be completely unimplementable on given hardware if the stricter alignment requirement is not met. However, that can never be true because falling back to a lock-based implementation is always possible.

The note could also be misinterpreted to imply that even though an object may be aligned to `required_alignment` and thus referenceable by an `atomic_ref`, operations could still fail to be lock-free because there is a stricter alignment requirement that the object does not meet. Such an interpretation is, however, at odds with p4.

The example given by the note is also confusing in that it does not necessarily demonstrate a situation in which `std::atomic_ref<T>::required_alignment` is greater than `alignof(T)`.

In conclusion, this note appears to be a convoluted way of saying that, in order to ensure that operations on `atomic_ref<T>` are lock-free, the implementation may define `required_alignment` to a value greater than `alignof(T)`. The note should be modified to say this much more clearly.

[2025-10-20; Reflector poll.]

Set status to Tentatively Ready after five votes in favour during reflector poll. Also ask SG1 to take a look.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 32.5.7.2 [\[atomics.ref.ops\]](#) as indicated:

```
static constexpr size_t required_alignment;
```

-1- The alignment required for an object to be referenced by an atomic reference, which is at least `alignof(T)`.

-2- [Note 1: ~~Hardware could require an object referenced by an `atomic_ref` to have stricter alignment (6.8.3 [\[basic.align\]](#)) than other objects of type `T`. Further, whether operations on an `atomic_ref` are lock-free could depend on the alignment of the referenced object. For example, lock-free operations on `std::complex<double>` could be supported only if aligned to $2 \times \text{alignof}(\text{double})$. An implementation can choose to define `atomic_ref<T>::required_alignment` to a value greater than `alignof(T)` in order to ensure that operations on all objects of type `atomic_ref<T>` are lock-free.~~ — end note]

4382. The `simd::basic_mask(bool)` overload needs to be more constrained

Section: 29.10.9.2 [\[simd.mask.ctor\]](#) Status: [Voting](#) Submitter: Matthias Kretz Opened: 2025-09-24 Last modified: 2025-10-30

Priority: Not Prioritized

Discussion:

29.10.9.2 [\[simd.mask.ctor\]](#) defines the overloads `basic_mask(bool)` and `basic_mask(unsigned_integral auto)`. This leads to the following pitfall:

```
auto g0() {
    unsigned short k = 0xf;
    return simd::mask<float, 8>(k); // mov eax, 15
}

auto g1() {
    unsigned short k = 0xf;
    return simd::mask<float, 8>(k >> 1); // mov eax, -1 🚩
}

auto g2() {
    unsigned int k = 0xf;
    return simd::mask<float, 8>(k >> 1); // mov eax, 7
}
```

In `g1`, `k` is promoted to `int`, shifted and then passed to the mask constructor. Instead of failing, `int(0x7)` is converted to `bool` and the mask thus initialized to all true.

Also consider:

1. `simd::mask<float>(true_type());`
2. `unsigned_integral<bool>` is `true` $\Rightarrow$ `same_as<bool> auto` instead of 'bool' makes the overload set ambiguous
3. `float` is convertible to `bool`, thus `simd::mask<float>(1.f)` continues to compile

Previous resolution [SUPERSEDED]:

This wording is relative to [N5014](#).

1. Modify 29.10.9.1 [\[simd.mask.overview\]](#), class template `basic_mask` synopsis, as indicated:

```
namespace std::simd {
    template<size_t Bytes, class Abi> class basic_mask {
    public:
```

```

    [...]

constexpr basic_mask() noexcept = default;

// 29.10.9.2 [simd.mask.ctor], basic_mask constructors
constexpr explicit basic_mask(value_type) noexcept;
template<size_t UBytes, class UAbi>
constexpr explicit basic_mask(const basic_mask<UBytes, UAbi>&) noexcept;
template<class G>
constexpr explicit basic_mask(G&& gen) noexcept;
constexpr basic_mask(const bitset<size()>& b) noexcept;
constexpr explicit basic_mask(unsigned_integral auto val) noexcept;
basic_mask(signed_integral auto) = delete;

    [...]
};
}

```

[2025-10-06; Matthias Kretz improves wording after reflector discussion]

[2025-10-23; Reflector poll.]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 29.10.9.1 [\[simd.mask.overview\]](#), class template basic_mask synopsis, as indicated:

```

namespace std::simd {
    template<size_t Bytes, class Abi> class basic_mask {
    public:
        [...]

        constexpr basic_mask() noexcept = default;

        // 29.10.9.2 [simd.mask.ctor], basic_mask constructors
        constexpr explicit basic_mask(same_as<value_type> auto) noexcept;
        template<size_t UBytes, class UAbi>
        constexpr explicit basic_mask(const basic_mask<UBytes, UAbi>&) noexcept;
        template<class G>
        constexpr explicit basic_mask(G&& gen) noexcept;
        template<same_as<bitset<size()>> T>
        constexpr basic_mask(const T& b) noexcept;
        template<unsigned_integral T> requires (!same_as<T, value_type>)
        constexpr explicit basic_mask(T unsigned_integral auto val) noexcept;

        [...]
    };
}

```

2. Modify 29.10.9.2 [\[simd.mask.ctor\]](#) as indicated:

```

constexpr explicit basic_mask(same_as<value_type> auto x) noexcept;

-1- Effects: Initializes each element with x.

[...]

template<same_as<bitset<size()>> T>
constexpr basic_mask(const T& b) noexcept;

-7- Effects: Initializes the  $i^{\text{th}}$  element with  $b[i]$  for all  $i$  in the range  $[0, \text{size}())$ .

template<unsigned_integral T> requires (!same_as<T, value_type>)
constexpr explicit basic_mask(T unsigned_integral auto val) noexcept;

-8- Effects: Initializes the first  $M$  elements to the corresponding bit values in val, [...]

```

4384. flat_set::erase(iterator) is underconstrained

Section: 23.6.11.2 [\[flat.set.defn\]](#), 23.6.12.2 [\[flat.multiset.defn\]](#) Status: [Voting](#) Submitter: Hewill Kang Opened: 2025-09-25 Last modified: 2025-10-30

Priority: Not Prioritized

Discussion:

This is a follow-up of LWG [3704](#)⁽¹⁾ since we now have flat_set and flat_multiset.

[2025-10-21; Reflector poll.]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 23.6.11.2 [\[flat.set.defn\]](#) as indicated:

```
iterator erase(iterator position) requires (!same_as<iterator, const iterator>);  
iterator erase(const_iterator position);
```

2. Modify 23.6.12.2 [\[flat.multiset.defn\]](#) as indicated:

```
iterator erase(iterator position) requires (!same_as<iterator, const iterator>);  
iterator erase(const_iterator position);
```

4398. enable_nonlocking_formatter_optimization should be disabled for container adaptors

Section: 23.6.2 [\[queue.syn\]](#), 23.6.5 [\[stack.syn\]](#) **Status:** [Voting](#) **Submitter:** Tomasz Kamiński **Opened:** 2025-10-02 **Last modified:** 2025-10-30

Priority: 2

Discussion:

As the standard currently defines formatters for queue, priority_queue, and stack enable_nonlocking_formatter_optimization is specialized to true for these adaptors per 28.5.6.4 [\[format.formatter.spec\]](#) p3:

Unless specified otherwise, for each type T for which a formatter specialization is provided by the library, each of the headers provides the following specialization:

```
template<> inline constexpr bool enable_nonlocking_formatter_optimization<T> = true;
```

However, formatting an adaptor requires formatting of the underlying range in terms of ranges::ref_view, and we disable the nonlocking_optimizations for all ranges, including ranges::ref_view.

This problem does not occur for the flat_set, flat_map adaptors, which are also ranges, but unlike stack etc. they do not have a specialized formatter. They use the formatter specialization for ranges and we already disable the optimization for that formatter.

The proposed wording has recently been implemented in [gcc's libstdc++](#).

[2025-10-14; Reflector poll]

Set priority to 2 after reflector poll.

This is a duplicate of LWG [4146^{\(i\)}](#), with a different proposed resolution.

[2025-10-17; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 23.6.2 [\[queue.syn\]](#), header <queue> synopsis, as indicated:

```
[...]  
// 23.6.13 \[container.adaptors.format\], formatter specialization for queue  
template<class charT, class T, formattable<charT> Container>  
    struct formatter<queue<T, Container>, charT>;  
  
template<class T, class Container>  
constexpr bool enable_nonlocking_formatter_optimization<queue<T, Container>> = false;  
  
// 23.6.4 \[priority.queue\], class template priority_queue  
template<class T, class Container = vector<T>,  
        class Compare = less<typename Container::value_type>>  
    class priority_queue;  
[...]  
// 23.6.13 \[container.adaptors.format\], formatter specialization for priority_queue  
template<class charT, class T, formattable<charT> Container, class Compare>  
    struct formatter<priority_queue<T, Container, Compare>, charT>;  
  
template<class T, class Container, class Compare>  
constexpr bool enable_nonlocking_formatter_optimization<priority_queue<T, Container, Compare>> = false;  
[...]
```

2. Modify 23.6.5 [\[stack.syn\]](#), header <stack> synopsis, as indicated:

```
[...]  
  
// 23.6.13 \[container.adaptors.format\], formatter specialization for stack  
template<class charT, class T, formattable<charT> Container>  
    struct formatter<stack<T, Container>, charT>;
```

```
template<class T, class Container>
constexpr bool enable_nonlocking_formatter_optimization<stack<T, Container>> = false;
```

[...]

4399. enable_nonlocking_formatter_optimization for pair and tuple needs remove_cvref_t

Section: 28.5.9 [\[format.tuple\]](#) Status: [Voting](#) Submitter: Tomasz Kamiński Opened: 2025-10-02 Last modified: 2025-10-30

Priority: Not Prioritized

Discussion:

The enable_nonlocking_formatter_optimization variable template is specialized only for cv-unqualified types. However, the specialization for pair and tuple does not remove the references and cv-qualifiers from the elements:

```
template<class... Ts>
constexpr bool enable_nonlocking_formatter_optimization<pair-or-tuple<Ts...>> =
    (enable_nonlocking_formatter_optimization<Ts> && ...);
```

As consequence pair<const std::string, int> or pair<const std::string&, int&> (map and flat_map reference types) will not use unbuffered prints.

The proposed wording has recently been implemented in [gcc's libstdc++](#).

[2025-10-17; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 28.5.9 [\[format.tuple\]](#) as indicated:

-1- For each of pair and tuple, the library provides the following formatter specialization where *pair-or-tuple* is the name of the template:

```
namespace std {
    [...]

    template<class... Ts>
    constexpr bool enable_nonlocking_formatter_optimization<pair-or-tuple<Ts...>> =
        (enable_nonlocking_formatter_optimization<remove_cvref_t<Ts>> && ...);
}
```

4403. simd::basic_vec CTAD misses difference type casting

Section: 29.10.7.2 [\[simd.ctor\]](#) Status: [Voting](#) Submitter: Hewill Kang Opened: 2025-10-04 Last modified: 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [simd.ctor].

View all other [issues](#) in [simd.ctor].

Discussion:

Currently, basic_vec can take an object r of range type R whose size is a constant expression and deduced to vec<ranges::range_value_t<R>, ranges::size(r)>.

However, such a deduced type is ill-formed when R has an integer-class type size which cannot be implicitly converted to *simd-size-type*, which is a signed integer type.

It is necessary to perform difference type casting here, and the narrowing conversion will still correctly be rejected due to the constructor's constraints.

[2025-10-17; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 29.10.7.2 [\[simd.ctor\]](#) as indicated:

```
template<class R, class... Ts>
    basic_vec(R&& r, Ts...) -> see below;
```

-17- *Constraints*:

(17.1) — R models `ranges::contiguous_range` and `ranges::sized_range`, and

(17.2) — `ranges::size(r)` is a constant expression.

-18- *Remarks*: The deduced type is equivalent to `vec<ranges::range_value_t<R>, static_cast<simd-size-type>(ranges::size(r))>`

4407. *constexpr-wrapper-like* needs `remove_cvref_t` in `simd::basic_vec` constructor

Section: 29.10.7.2 [[simd.ctor](#)] **Status:** [Voting](#) **Submitter:** Hewill Kang **Opened:** 2025-10-05 **Last modified:** 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [[simd.ctor](#)].

View all other [issues](#) in [[simd.ctor](#)].

Discussion:

`decltype(From::value)` would be `const int&` if `From` is a type of `std::cw<42>`, so the reference also needs to be removed for checking the arithmetic type.

[2025-10-17; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 29.10.7.2 [[simd.ctor](#)] as indicated:

```
template<class U> constexpr explicit(see below) basic_vec(U&& value) noexcept;
```

-1- Let `From` denote the type `remove_cvref_t<U>`.

[...]

-4- *Remarks*: The expression inside `explicit` evaluates to `false` if and only if `U` satisfies `convertible_to<value_type>`, and either

(4.1) — `From` is not an arithmetic type and does not satisfy *constexpr-wrapper-like*,

(4.2) — `From` is an arithmetic type and the conversion from `From` to `value_type` is value-preserving (29.10.1 [[simd.general](#)]), or

(4.3) — `From` satisfies *constexpr-wrapper-like*, `remove_cvref_t remove_const_t<decltype(From::value)>` is an arithmetic type, and `From::value` is representable by `value_type`.

4412. Fix declaration of `zero_element` and `uninit_element`

Section: 29.10.3 [[simd.syn](#)] **Status:** [Voting](#) **Submitter:** Jonathan Wakely **Opened:** 2025-10-17 **Last modified:** 2025-10-30

Priority: Not Prioritized

View all other [issues](#) in [[simd.syn](#)].

Discussion:

Addresses US-174-282

`zero_element` and `uninit_element` should be `inline` and not `static`

[2025-10-22; Reflector poll.]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 29.10.3 [[simd.syn](#)] as indicated:

```
// 29.10.8.7, permute by static index generator
inline static constexpr simd-size-type zero_element = implementation-defined;
inline static constexpr simd-size-type uninit_element = implementation-defined;
```

4413. Unused/left-over `simd::alignment` specialization for `basic_mask`

Section: 29.10.4 [[simd.traits](#)] Status: [Voting](#) Submitter: Matthias Kretz Opened: 2025-10-15 Last modified: 2025-10-30

Priority: Not Prioritized

View all other [issues](#) in [[simd.traits](#)].

Discussion:

29.10.4 [[simd.traits](#)] describes a value member for `simd::alignment<basic_mask<...>, bool>`. This was used for loads and stores for masks from/to arrays of `bool`. However, these load/store functions were removed after `bitset/unsigned_integral` conversions were introduced. This left-over TS wording should be removed.

[2025-10-22; Reflector poll.]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 29.10.3 [[simd.syn](#)] as indicated:

```
template<class T, class U = typename T::value_type> struct alignment { see below };
```

-1- `alignment<T, U>` has a member value if and only if

(1.1) — `T` is a specialization of `basic_mask` and `U` is `bool`, or

(1.2) — `T` is a specialization of `basic_vec` and `U` is a vectorizable type.

4415. `task::promise_type::uncaught_exception` seems to be misnamed

Section: 33.13.6.5 [[task.promise](#)] Status: [Voting](#) Submitter: Jiang An Opened: 2025-10-17 Last modified: 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [[task.promise](#)].

View all other [issues](#) in [[task.promise](#)].

Discussion:

According to 9.6.4 [[dcl.fct.def.coroutine](#)], a function of name `unhandled_exception` may be called by the language mechanisms. However, `std::execution::task<T, Environment>::promise_type` has an `uncaught_exception` function instead, which won't be implicitly called.

In [P3552R3](#), `unhandled_exception` was discussed, but the wording specified `uncaught_exception`, which looks like a mistake. Moreover, the paper didn't talk about the status of `uncaught_exception` in the zombie name list ([`tab:zombie.names.std`]). Perhaps `unhandled_exception` is the correct function name.

[2025-10-23; Reflector poll.]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 33.13.6.5 [[task.promise](#)] as indicated:

```
namespace std::execution {
    template<class T, class Environment>
    class task<T, Environment>::promise_type {
    public:
        [...]
        void uncaughtunhandled_exception();
        coroutine_handle<> unhandled_stopped();
        [...]
    };
}
```

[...]

```
void uncaughtunhandled_exception();
```

-12- Effects: If the signature `set_error_t(exception_ptr)` is not an element of `error_types`, calls `terminate()` (14.6.2 [[except.terminate](#)]). Otherwise, stores `current_exception()` into `errors`.

4416. `<meta>` should include `<compare>`

Section: 21.4.1 [\[meta.syn\]](#) **Status:** [Voting](#) **Submitter:** Jiang An **Opened:** 2025-10-15 **Last modified:** 2025-10-30

Priority: Not Prioritized

Discussion:

Some inclusions from <meta> were removed between [P2996R7](#) and [P2996R8](#). However, given std::meta::member_offsets has operator<=>, perhaps we should still include <compare> in <meta>. This is also noticed in [P3429R1](#).

[2025-10-20; Reflector poll.]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 21.4.1 [\[meta.syn\]](#), header <meta> synopsis, as indicated:

```
#include <compare> // see 17.12.1 \[compare.syn\]
#include <initializer_list> // see 17.11.2 \[initializer.list.syn\]

namespace std {
    [...]
}
```

[4422](#). meta::access_context should be a consteval-only type

Section: 21.4.8 [\[meta.reflection.access.context\]](#) **Status:** [Voting](#) **Submitter:** Jakub Jelinek **Opened:** 2025-10-20 **Last modified:** 2025-10-30

Priority: 2

Discussion:

The meta::access_context type is expected to contain some meta::info objects, which would make it a consteval-only type. But we don't actually specify any members, so nothing in the current specification says you can't persist one until runtime.

[2025-10-23; Reflector poll. Adjust proposed wording.]

Set priority to 2 after reflector poll.

Reflector discussion requested that 'non-aggregate' and 'consteval-only' both be put in paragraph 3, adjacent to 'structural'. Also added a drive-by editorial change to paragraph 1.

[2025-10-23; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 21.4.8 [\[meta.reflection.access.context\]](#) as indicated:

-1- The `class` access_context ~~class is a non-aggregate type that~~ represents a namespace, class, or function from which queries pertaining to access rules may be performed, as well as the designating class (11.8.3 [\[class.access.base\]](#)), if any.

-2- An access_context has an associated scope and designating class.

```
namespace std::meta {
    struct access_context {
        access_context() = delete;

        consteval info scope() const;
        consteval info designating_class() const;

        static consteval access_context current() noexcept;
        static consteval access_context unprivileged() noexcept;
        static consteval access_context unchecked() noexcept;
        consteval access_context via(info cls) const;
    };
}
```

-3- access_context is a structural, `consteval-only, non-aggregate` type. Two values ac1 and ac2 of type access_context are template-argument-equivalent (13.6 [\[temp.type\]](#)) if ac1.scope() and ac2.scope() are template-argument-equivalent and ac1.designating_class() and ac2.designating_class() are template-argument-equivalent.

[4425](#). CTAD function_ref of data member pointer should produce noexcept signature

Section: 22.10.17.6.5 [[func.wrap.ref.deduct](#)] Status: [Voting](#) Submitter: Tomasz Kamiński Opened: 2025-10-20 Last modified: 2025-10-30

Priority: Not Prioritized

Discussion:

Addresses PL-005.

For data member pointer M C::* dmp, template argument deduction in form `std::function_ref fr(std::nontype<dmp>, x)` results in `std::function_ref<M&()>`, that returns a reference to designated member. As accessing a data member can never throw exception and `function_ref` support noexcept qualified function types, the deduction guide should be updated to produce noexcept-qualified signature.

[2025-10-21; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 22.10.17.6.5 [[func.wrap.ref.deduct](#)] as indicated:

```
template<class F>
function_ref(nontype_t<f>, T&&) -> function_ref<see below>;
```

Let F be `decltype(f)`.

-6- Constraint:

- (6.1) — F is of the form `R(G::*)(A...) cv &opt noexcept(E)` for type G, or
- (6.2) — F is of the form `M G::*` for type G and object type M, in which case let R be `invoke_result_t<F, T&>`, A... be an empty pack, and E be `false` **true**, or
- (6.3) — F is of the form `R(*) (G, A...) noexcept(E)` for type G.

-7- Remarks: The deduced type is `function_ref<R(A...) noexcept(E)>`.

[4426](#). Clarify what `meta::reflect_constant_string` considers a string literal

Section: 21.4.15 [[meta.reflection.array](#)] Status: [Voting](#) Submitter: Jakub Jelinek Opened: 2025-10-21 Last modified: 2025-10-30

Priority: Not Prioritized

View other [active issues](#) in [meta.reflection.array].

View all other [issues](#) in [meta.reflection.array].

Discussion:

`meta::reflect_constant_string` says:

Let V be the pack of values of type CharT whose elements are the corresponding elements of r, except that if r refers to a string literal object, then V does not include the trailing null terminator of r.

It's unclear how the implementation should decide whether r refers to a string literal. If R models `contiguous_iterator`, should it use `meta::is_string_literal(ranges::data(r))`? Should it omit the `'\0'` from `string_view("abc", 3)`?

Also, "null terminator" is only defined in 27.4.3.1 [[basic.string.general](#)] and not used for string literal objects (5.13.5 [[lex.string](#)]).

[2025-10-23; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 21.4.15 [[meta.reflection.array](#)] as indicated:

```
template<ranges::input_range R>
constexpr info reflect_constant_string(R&& r);
```

-2- Let CharT be `ranges::range_value_t<R>`.

-3- Mandates: CharT is one of `char`, `wchar_t`, `char8_t`, `char16_t`, `char32_t`.

Let V be the pack of values of type CharT whose elements are the corresponding elements of r, except that if r **refers is a** **reference** to a string literal object, then V does not include the **trailing null terminator** **terminating u+0000 NULL character** of r.